AFEKA
The Academic College of Engineering in Tel Aviv

Afeka College of Engineering

Department of Software Engineering

Ambient Invisible Intelligence - Spring 2026

Final Project Progress Report

Smart Parking Management System

SPOT&SLOT

"The Moovit of Parking"

Sprint 4

Due Date: 08/07/2026 08:00, Wednesday

Our Team - The Penguins:

Name	Avatar in Kanban Board	Role
Omer Asayag		Team + Team Lead + Scrum Master
Avishay Haimov		Team + Product Owner
Tom Gurevich		Team + QA Engineer + Configuration Manager
Stephanie Kovaks		Team + Technical Writer + UI/UX Engineer
Shawn Farberov		Team + System Architect
Nicole Barghout		Team + DBA + DevOps

AFEKA
The Academic College of Engineering in Tel Aviv

Table of Contents

Sketch of DB **4**

The System Flow **5-6**

Software Requirements Specification **7**

1. Introduction 8
    1.1 Project Overview 8
    1.2 Goals 8
    1.3 Scope 8
2. Actors 9
3. Functional Requirements 10
    3.1 Use Cases by Actor 10
    3.2 Driver Use Cases 11
    3.3 Operator Use Cases 14
    3.4 Administrative Operations (Admin API) 15
    3.5 System Operations 15
4. Non-Functional Requirements 16
5. Domain Object Types 17
6. Commands per Object Type 17
7. REST API, Roles & Authentication (Sprint 4) 18
    7.1 Role Permission Matrix 18
    7.2 Custom Search Queries 18
    7.3 Example Requests 18
8. System Architecture & Deployment 18

Technologies Appendix **19**

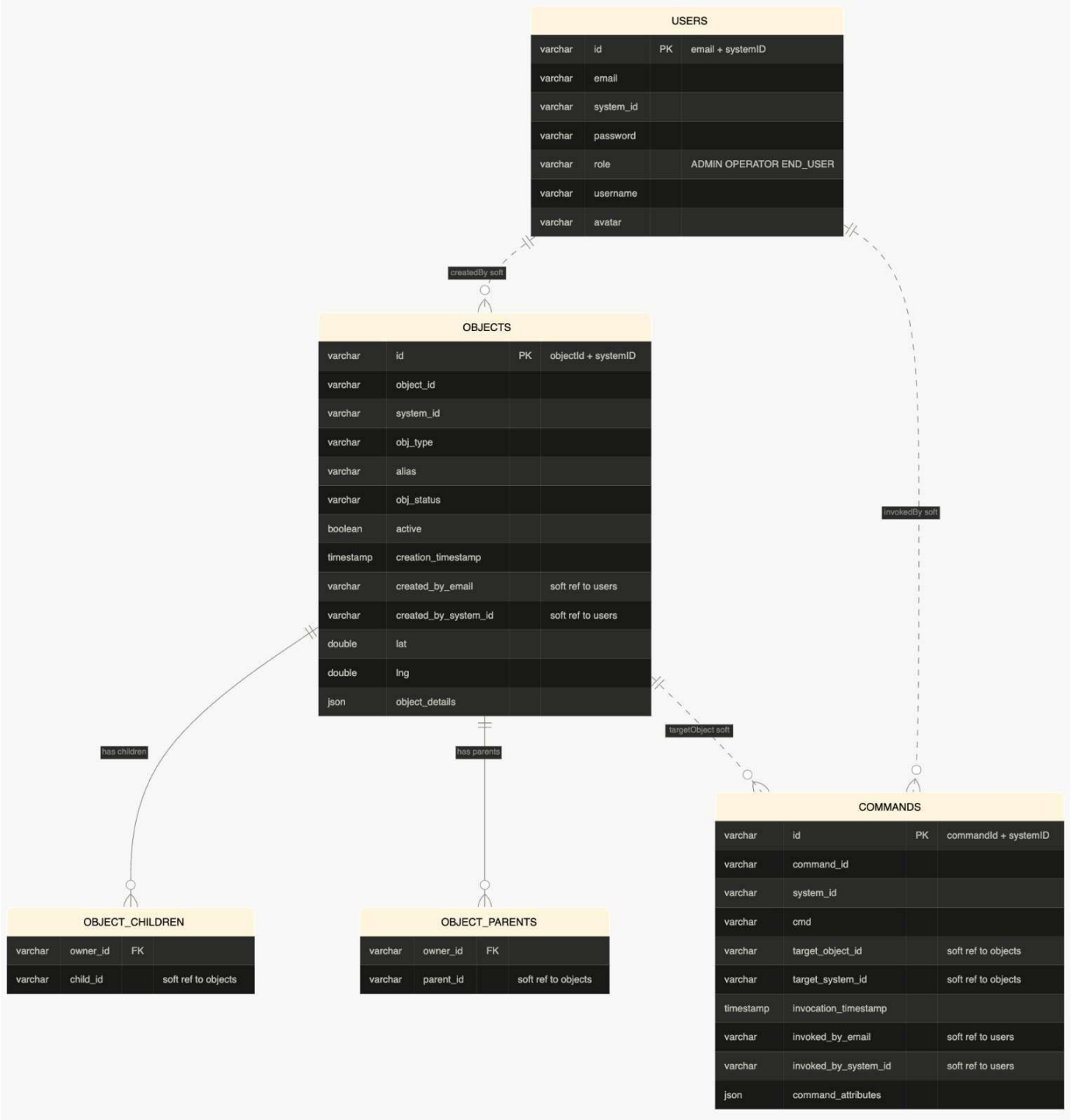
1. Overview 19
2. Server — Smart Parking API 19
3. Server — Testing 19
4. Clients — Mobile App & Staff Console 20
5. MachineLearning Microservice 20
6. Infrastructure & Tooling 20

Software Test Report **21**

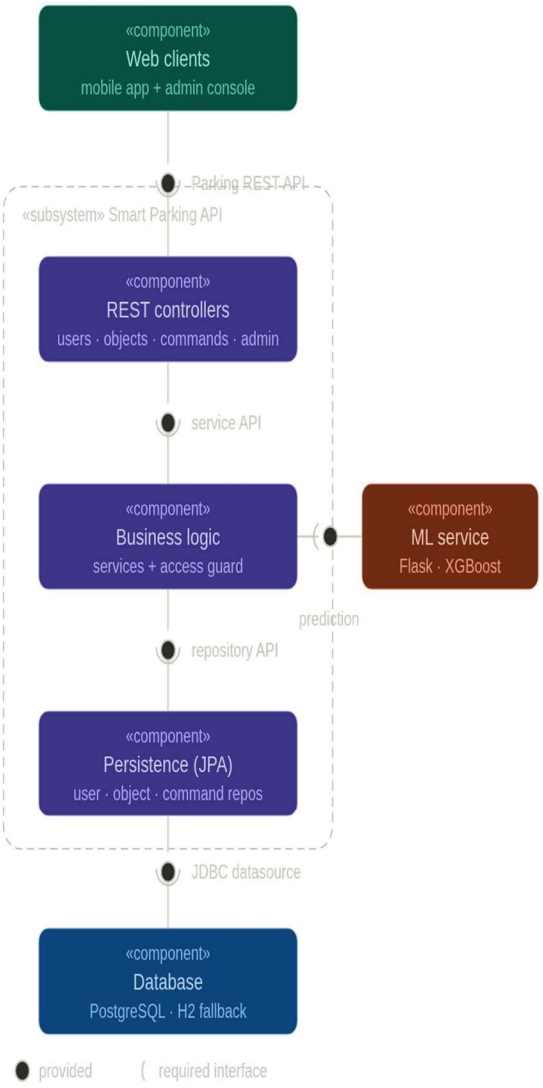
1. Test Approach 21
2. Roles & Credentials Used in Tests 21
3. Test Scenarios 21
4. Performance Testing (Apache JMeter) 29

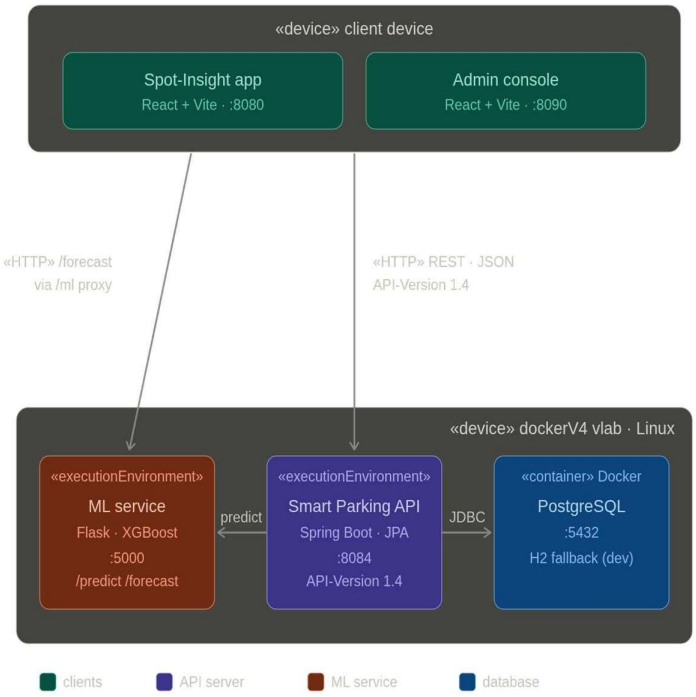
Spot&Slot – UX/UI Research **31**
1. Summary of Research Findings – Target Audience Characteristics 32
2. Product Goal – Addressing the Following Problems 35
3. Product Usage Environment 36
4. Survey Execution and Insights 37
5. Persona Definition and User Audience 41
6. Summary of Interview Findings 44
7. Future Use Cases (Scenarios) 45
8. Competitor Analysis 45
Feedback & Refinement of The Interface 47
9. Extended Functionality – UX Research & Rationale 50
Bibliography 53
Model Evaluation Report **54**
1. Introduction & Objective 55
2. Dataset 55
2.1 Parking lots 55
2.2 Features & special events 55
3. Methodology 56
4. Exploratory Data Analysis 56
5. Model Comparison 59
6. Best Model — XGBoost 59
6.1 Feature importance 59
6.2 Prediction quality & residuals 60
6.3 Hyper-parameter tuning 61
7. Conclusion & Recommendation 61
Project Progress Throughout Kanban Boards **62**
Sprint 1 62
Sprint 2 63
Sprint 3 64
Sprint 4 68
General Summary of Project's Work Process 72
Progress Throughout The Kanban Board – Sprint 4 74
General Summary of Work in Sprint 4 78

SKETCH OF DB:


USERS
varchar id PK email + systemID
varchar email
varchar system_id
varchar password
varchar role ADMIN OPERATOR END_USER
varchar username
varchar avatar
createdBy soft
invokedBy soft
OBJECTS
varchar id PK objectId + systemID
varchar object_id
varchar system_id
varchar obj_type
varchar alias
varchar obj_status
boolean active
timestamp creation_timestamp
varchar created_by_email soft ref to users
varchar created_by_system_id soft ref to users
double lat
double lng
json object_details
has children
has parents
targetObject soft
COMMANDS
varchar id PK commandId + systemID
varchar command_id
varchar system_id
varchar cmd
varchar target_object_id soft ref to objects
varchar target_system_id soft ref to objects
timestamp invocation_timestamp
varchar invoked_by_email soft ref to users
varchar invoked_by_system_id soft ref to users
json command_attributes
OBJECT_CHILDREN
varchar owner_id FK
varchar child_id soft ref to objects
OBJECT_PARENTS
varchar owner_id FK
varchar parent_id soft ref to objects

The system flow




«device» client device
Spot-Insight app
React + Vite · :8080
Admin console
React + Vite · :8090
«HTTP» /forecast
via /ml proxy
«HTTP» REST · JSON
API-Version 1.4
«device» dockerV4 vlab · Linux
«executionEnvironment»
ML service
Flask · XGBoost
:5000
/predict /forecast
predict
«executionEnvironment»
Smart Parking API
Spring Boot · JPA
:8084
API-Version 1.4
JDBC
«container» Docker
PostgreSQL
:5432
H2 fallback (dev)
clients
API server
ML service
database

AFEKA
The Academic College of Engineering in Tel Aviv

Afeka College of Engineering

Department of Software Engineering

Ambient Invisible Intelligence - Spring 2026

Software Requirements Specification

Smart Parking Management System

Spot&Slot

"The Moovit of Parking"

System ID: 2026b.Omer.Asayag

API Version: 1.4 · Sprint 4 (Final)

1. Introduction

1.1 Project Overview

Spot&Slot is a Smart Parking Management System built as an Ambient Invisible Intelligence (AII) project, following Gartner's model in which intelligence is embedded invisibly into everyday infrastructure: drivers simply open the app and receive live, predicted parking information without being aware of the sensing, modelling and serving that happens behind the scenes.

The system follows a client/server architecture. The server is the main project, a Spring Boot application that provides real-time parking management for urban parking lots: it tracks parking-slot availability across lots and sections, monitors slot health, authenticates callers, and uses a machine-learning microservice to predict future availability. It exposes a RESTful API (base path /ambient-invisible-intelligence) that clients consume.

Two clients are built on top of the API:

- Spot&Slot Mobile: a driver-facing app that runs on Android (packaged natively with Capacitor) and in the browser. It shows the live parking lot, lets the driver choose which lot to view, and displays server-backed availability forecasts.
- Spot&Slot Console: a desktop web application for staff. Operators build and manage the parking objects they own (lots, sections, slots); administrators manage users and review command history via the Admin API.

The project is built with Spring Boot 4.1 on JDK 25, backed by PostgreSQL (via Spring Data JPA) for persistent storage, and follows the AII course specification for its REST contract (API version 1.4).

1.2 Goals

- Business: reduce the time drivers spend searching for parking by providing accurate, up-to-date availability data and future-availability predictions.
- Business: give parking-lot operators tools to manage their lots' structure and state, and give administrators oversight of users and system activity.
- Team: practise a clean client/server architecture with a well-defined, versioned REST API, automated testing, role-based access control, and a machine-learning microservice, per the AII course specification.

1.3 Scope

In scope:

- Server-side REST API (v1.4) implementing the course specification, including custom search queries, pagination, and password-authenticated, role-based operations.
- CRUD on users, objects and commands; smart-parking domain logic (availability, best-section, health checks, slot-state, ML prediction).
- Persistent storage with PostgreSQL (run in Docker), managed through Spring Data JPA.
- ML-based availability prediction via a Python (Flask + XGBoost) microservice.
- Spot&Slot Mobile: a native Android driver app (Capacitor) plus a browser build: live lot map, lot selector, and ML forecast view.
- Spot&Slot Console: a desktop web app for operators (manage owned lots/sections/slots) and administrators (manage users, review activity).

Out of scope:

- Hardware sensors or embedded devices; slot status is updated through the API.
- Payment processing or billing.
- iOS native application — the mobile app targets Android and the browser; iOS remains future work.

2. Actors

Access to operations is governed by the user's role. Three human roles and one supporting actor interact with the system.

Actor (Role)	Type	Description	Goals
Driver (END_USER)	Primary	An end user who finds and plans parking through the mobile app. Personas include, for example, Maya, a student commuting to Tel Aviv University, and Dana, a parent driving to the Port on weekends.	Find available parking quickly; see accurate live status and forecasts for a chosen lot; be notified when a spot frees up.
Operator (OPERATOR)	Primary	Staff responsible for the parking infrastructure they manage. Operators create and maintain the lots, sections and slots they own, and keep slot state current.	Keep the structure and live state of their lots accurate; browse and search all objects without restriction.
Administrator (ADMIN)	Primary	A system administrator. Administrators manage user accounts and review system activity through the Admin API.	Manage users; audit the command history; purge data during maintenance.
<<Time>>	Supporting	A scheduled trigger that fires periodic background tasks (slot-health monitoring).	—

Note: Driver identification is mandatory for every driver operation. Because the credentials (userEmail + userPassword, verified against the stored account) are supplied on every request, each Driver use case below begins its flow with the system authenticating the driver.

3. Functional Requirements

3.1 Use Cases by Actor

Figure 1 shows the use-case diagram for the implemented system. Per the Sprint 3 review, the diagram is limited to the functional (user-facing) behaviour of the Driver and Operator actors. Administrative maintenance (manage users, review command history, purge data) and automated slot-health monitoring are non-functional, they are served by the Admin API and a scheduled «Time» task, and are documented in sections 3.4 and 3.5 but are deliberately not drawn in the functional use-case diagram. The use cases realised in the final system are:

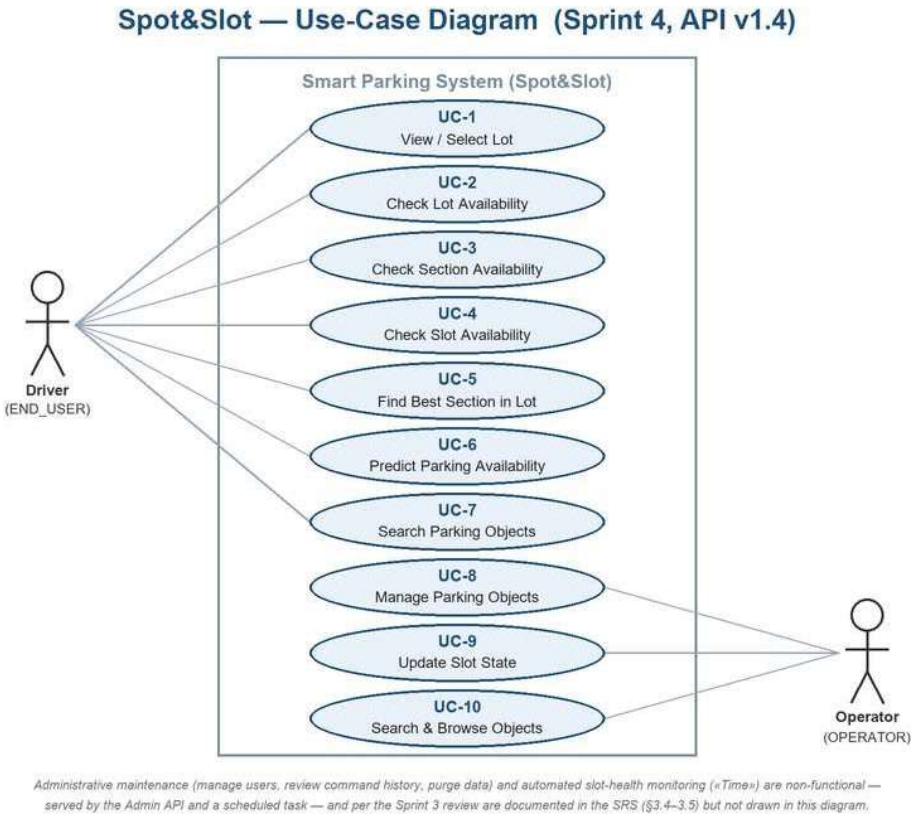


Figure 1 — Spot&Slot functional use-case diagram (Driver & Operator; Sprint 4, API v1.4)

Actor	Use Cases	Notes
Driver (END_USER)	UC-1 View / Select Lot UC-2 Check Lot Availability UC-3 Check Section Availability UC-4 Check Slot Availability UC-5 Find Best Section in Lot UC-6 Predict Parking Availability UC-7 Search Parking Objects	Reads return only active objects. Commands may be invoked only by a Driver (END_USER).
Operator (OPERATOR)	UC-8 Manage Parking Objects UC-9 Update Slot State UC-10 Search & Browse Objects	Operators own the lots they create; they may modify only their own lots' objects, and may read/search everything.
Administrator (ADMIN)	AO-1 Manage Users AO-2 Review Command History AO-3 Purge Data	Admins use only the Admin API.
<<Time>>	SO-1 Monitor Slot Health	Scheduled background task.

3.2 Driver Use Cases

UC-1: View / Select Lot

Primary Actor: Driver (END_USER)

Precondition: The driver has an END_USER account; at least one active PARKING_LOT exists.

Basic Flow:

1. The driver opens the mobile app;
2. the system attempts to authenticate the driver from the supplied credentials.
3. the system succeeds in authenticating the driver from the supplied credentials
4. The system lists the active parking lots and the driver selects one to view.
5. The system returns the selected lot with its sections and slots and renders the live map.

Alternate Flows:

- A1 — Authentication fails: the system attempts to authenticate the driver from the supplied credentials but the credentials are missing or the password does not match the stored account; the system returns HTTP 401 and the app shows a sign-in error. Expected result: the driver is not admitted.
- A2 — No active lots exist: the system succeeds in authenticating the driver from the supplied credentials and the system returns an empty list; the app shows an empty state.

Postcondition: The driver is viewing an active lot's live status.

UC-2: Check Lot Availability

Primary Actor: Driver (END_USER)

Precondition: An active PARKING_LOT with slots exists.

Basic Flow:

6. The system successfully authenticates the driver.
7. The driver invokes availabilityInLot on a PARKING_LOT, optionally with a slotKindFilter (e.g. HANDICAP).
8. The system attempts to verify the target lot exists and is active, counts active FREE slots matching the filter, and counts SEARCHING vehicles in the lot.
9. the system successfully verifies the target lot exists and is active
10. The system returns freeSlots, searchingDrivers and worthEntering (true when freeSlots > searchingDrivers). Expected result: an availability summary for the lot.

Alternate Flows:

- A1 — Not an END_USER: the system attempts to authenticate the user is a driver but sees the user has another role; the system returns HTTP 403.
- A2 — Target missing or inactive: The system successfully authenticates the driver; the driver invokes availabilityInLot on a PARKING_LOT; the system discovers that the target is missing or inactive; the system treats it as absent and returns HTTP 404.
- A3 — slotKindFilter supplied: The system successfully authenticates the driver; the driver invokes availabilityInLot on a PARKING_LOT with slotKindFilter HANDICAP; the system successfully verifies the target lot exists and is active with the added filter; only slots of that kind are counted.

Postcondition: None (read-only).

UC-3: Check Section Availability

Primary Actor: Driver (END_USER)

Precondition: An active PARKING_SECTION with slots exists.

Basic Flow:

11. The system successfully authenticates the driver.
12. The driver invokes availabilityInSection on a PARKING_SECTION.
13. The system searches for the section and once it verifies it exists counts the free and total active slots in the section and returns freeSlots and totalSlots.

Alternate Flows:

- A1 — Not an END_USER: the system attempts to authenticate the user is a driver but sees the user has another role; the system returns HTTP 403.
- A2 — Target missing or inactive: The system successfully authenticates the driver; the driver invokes availabilityInSection on a PARKING_SECTION; the system discovers that the target is missing or inactive; the system treats it as absent and returns HTTP 404.
- A3 — slotKindFilter supplied: The system successfully authenticates the driver; the driver invokes availabilityInSection on a PARKING_SECTION with slotKindFilter HANDICAP; the system successfully verifies the target section exists and is active with the added filter; only slots of that kind are counted.
-

Postcondition: None (read-only).

UC-4: Check Slot Availability

Primary Actor: Driver (END_USER)

Precondition: An active PARKING_SLOT exists.

Basic Flow:

14. The system successfully authenticates the driver.
15. The driver invokes checkSlotAvailability on a PARKING_SLOT.
16. The system searches for the slot and once it verifies it exists returns slotId, alias, status, isFree, slotKind and healthStatus.

Alternate Flows:

- A1 — Not an END_USER: the system attempts to authenticate the user is a driver but sees the user has another role; the system returns HTTP 403.
- A2 — Slot missing or inactive: The system successfully authenticates the driver; the driver invokes checkSlotAvailability on a PARKING_SLOT; the system discovers that the target is missing or inactive; the system treats it as absent and returns HTTP 404.(an inactive slot is reported as if it did not exist).

Postcondition: None (read-only).

UC-5: Find Best Section in Lot

Primary Actor: Driver (END_USER)

Precondition: An active PARKING_LOT with multiple sections exists.

Basic Flow:

17. The system successfully authenticates the driver.
18. The driver invokes bestSectionInLot on a PARKING_LOT, optionally with a slotKindFilter.
19. The system verifies that the parking lot is not full and groups free slots by section and identifies the section with the most free slots.
20. The system returns bestSectionId, its freeSlots count and a per-section breakdown.

Alternate Flows:

- A1 — Not an END_USER: the system attempts to authenticate the user is a driver but sees the user has another role; the system returns HTTP 403.
- A2 — All sections full: The system successfully authenticates the driver; The driver invokes bestSectionInLot on a PARKING_LOT, optionally with a slotKindFilter; the system discovers that the parking lot is full; the system returns bestSectionId is null and freeSlots is 0.
- A3 — Target missing or inactive: The system successfully authenticates the driver; the driver invokes bestSectionInLot on a PARKING_LOT, optionally with a slotKindFilter; the system discovers that the target is missing or inactive; the system treats it as absent and returns HTTP 404.
  the system discovers that the target is missing or inactive; the system treats it as absent and returns HTTP 404.

Postcondition: None (read-only).

UC-6: Predict Parking Availability

Primary Actor: Driver (END_USER)

Precondition: An active PARKING_LOT exists; the ML prediction microservice is deployed.

Basic Flow:

21. The system successfully authenticates the driver.
22. The driver invokes predictAvailability on a PARKING_LOT with a targetTimestamp.
23. The system verifies that the parking lot is not full and computes the lot's current free slots and total capacity and calls the Python ML service.
24. The ML service returns a predicted occupancy that the system scales to the lot capacity, caps to [0, totalSlots], and returns.

Alternate Flows:

- A1 — Not an END_USER: the system attempts to authenticate the user is a driver but sees the user has another role; the system returns HTTP 403.
- A2 — ML service unreachable: The system successfully authenticates the driver; The driver invokes predictAvailability on a PARKING_LOT with a targetTimestamp; the system verifies the parking lot is not full and attempts to call the ML service; falls back to the current free-slot count and returns a fallback note (graceful degradation).
- A3 — targetTimestamp missing: The system successfully authenticates the driver; the driver invokes predictAvailability on a PARKING_LOT without a targetTimestamp; the system returns the current free slots with an informational note.

Postcondition: None (read-only).

UC-7: Search Parking Objects

Primary Actor: Driver (END_USER)

Precondition: The driver is authenticated.

Essential purpose: let a driver find lots and slots by name, type, status, or geographic proximity, returning results a page at a time.

Basic Flow:

25. The system successfully authenticates the driver.
26. The driver issues a search — by exact alias, by alias pattern, by type, by status, or by location (a square, or optionally a circle, around a lat/lng within a given distance in KM/MILES/NEUTRAL units) — with size and page parameters.
27. The system returns the matching active objects for the requested page.

Alternate Flows:

- A1 — Inactive matches: for a driver, inactive objects are never returned — they are treated as if absent.

- A2 — Invalid units or page size: the system returns HTTP 400.

Postcondition: None (read-only).

3.3 Operator Use Cases

UC-8: Manage Parking Objects

Primary Actor: Operator (OPERATOR)

Precondition: The operator is authenticated as OPERATOR.

Basic Flow:

28. The system successfully authenticates the operator.
29. The operator creates a PARKING_LOT (with a required non-blank status); the system records the operator as the lot's owner (createdBy).
30. The operator adds PARKING_SECTIONs and PARKING_SLOTs under that lot, and may update or delete them after the system successfully verifies that the operator is the owner of this lot; deleting a lot cascades to its sections and slots.
31. The operator may bind objects into parent/child relationships after the system verifies ownership.

Alternate Flows:

- A1 — Not an OPERATOR: the system attempts to authenticate the user is an operator but sees the user has another role; creating/updating/deleting objects is refused with HTTP 403.
- A2 — Not the owner: The system successfully authenticates the operator; an operator attempting to modify another operator's lot (or its children); the system attempts to verify ownership and receives HTTP 403.
- A3 — Missing required field (type, alias or status is null/blank): The system successfully authenticates the operator; the operator attempts to create a PARKING_LOT / PARKING_SECTION / PARKING_SLOT with missing fields; the system returns HTTP 400.

Postcondition: The operator's parking objects are created/updated/deleted as requested.

UC-9: Update Slot State

Primary Actor: Operator (OPERATOR)

Precondition: A PARKING_SLOT owned by the operator exists.

Basic Flow:

32. The system successfully authenticates the operator.
33. The operator sets the slot's status to FREE or OCCUPIED through the object-update operation (PUT /objects); the system verifies that the operator owns the lot and updates the slot's status. This operator path (PUT /objects) is distinct from the END_USER updateSlotState command in section 6.

Alternate Flows:

- A1 — Not the owning operator: The system successfully authenticates the operator; The operator sets the slot's status to FREE or OCCUPIED; the system sees the operator is not the owner and returns HTTP 403.
- A2 — Blank status: The system successfully authenticates the operator; the operator attempts to set the status to blank; the system returns HTTP 400.

Postcondition: The slot's status is updated.

UC-10: Search & Browse Objects

Primary Actor: Operator (OPERATOR)

Precondition: The operator is authenticated.

Basic Flow:

34. The system successfully authenticates the operator.
35. The operator lists or searches objects (same queries as UC-7), with pagination.
36. The system returns matching objects — including inactive ones, which operators may see.

Postcondition: None (read-only).

3.4 Administrative Operations (Admin API)

AO-1: Manage Users

Primary Actor: Administrator (ADMIN)

Basic Flow:

37. The system successfully authenticates the administrator (ADMIN role required).
38. The administrator exports all users (paginated). Creating an elevated account (ADMIN/OPERATOR) is authorised by an existing admin; public sign-up creates END_USER accounts.

Alternate Flow:

- A1 — Not an ADMIN: the system attempts to authenticate the user is an admin but sees the user has another role; returns HTTP 403.

Postcondition: Administrator has the user list / a new account exists.

AO-2: Review Command History

Primary Actor: Administrator (ADMIN)

Basic Flow:

39. The system successfully authenticates the administrator.
40. The administrator exports the command history (paginated).

Postcondition: Administrator has an audit of invoked commands.

AO-3: Purge Data

Primary Actor: Administrator (ADMIN)

Basic Flow:

41. The system successfully authenticates the administrator.
42. The administrator deletes all users, all objects (including their relations), or all commands.

Postcondition: The selected data store is cleared.

3.5 System Operations

SO-1: Monitor Slot Health

Primary Actor: <<Time>>

Precondition: The system is running; slots carry lastHeartbeat timestamps.

Basic Flow:

43. The scheduled task fires every 30 seconds.
44. The system scans PARKING_SLOT objects; for each slot whose heartbeat is older than the staleness threshold and is not already FAULTY, it sets healthStatus to FAULTY.

Postcondition: Stale slots are automatically marked FAULTY.

4. Non-Functional Requirements

#	Category	Requirement	Acceptance Criterion
1	Security	Every non-account operation shall authenticate the caller by verifying a supplied password against the stored account, and shall authorise the action against the caller's role (ADMIN / OPERATOR / END_USER).	A wrong password returns HTTP 401; a valid user acting outside their role returns HTTP 403 — as verified by the integration and BDD tests.
2	Reliability	The system shall degrade gracefully when the ML microservice is unavailable: predictAvailability shall still return a result based on the current free-slot count.	With the ML service stopped, predictAvailability returns HTTP 200 with a fallback note (verified by test).
3	Reliability	The health-monitoring scheduler shall mark a slot FAULTY within one scheduler cycle of its heartbeat expiring.	A slot whose lastHeartbeat exceeds the threshold is FAULTY within 30 s of the next cycle.
4	Maintainability	Every list/search operation shall support pagination (size, page); non-paginated multi-object endpoints are deprecated and not served.	All list/search endpoints accept size and page and return the requested page; a review of the controllers confirms no non-paginated multi-fetch remains.
5	Portability	The server shall run against PostgreSQL started via Docker Compose, and shall be configurable for either a clean-on-restart development database or a persistent production database via Spring profiles.	The app boots with the default profile (schema recreated) and with the prod profile (schema preserved); both are documented in the READMEs.
6	Usability	Error responses shall be machine-readable so client developers can handle failures programmatically: every invalid, unauthorised or not-found request returns the appropriate HTTP status (400/401/403/404) with a JSON error body.	Each error path returns the documented status and a JSON body in Spring's standard format (status, error, path), as verified by tests.

5. Domain Object Types

Domain data is stored as ObjectBoundary instances. Every object carries an identity (objectId + systemID), a required non-blank status, an active flag (inactive objects are hidden from END_USERs), and a createdBy owner.

Object Type	Description	Key objectDetails Fields
PARKING_LOT	A physical parking lot with a name (alias) and city; may carry a geo location (lat/lng) used by the location search.	address (city)
PARKING_SECTION	A named section within a lot (e.g. Section A).	parentLotId
PARKING_SLOT	An individual slot with availability status, kind and embedded health data.	parentLotId, parentSectionId, slotKind, healthStatus, lastHeartbeat
VEHICLE	An event record for a vehicle action (entering, exiting, searching).	lotId, driverEmail, entryTime, plate

6. Commands per Object Type

Each command is bound to a target object type; invoking it on the wrong type returns an error. Commands may be invoked only by an END_USER, and only on an object that exists and is active.

Target Type	Command	Description
PARKING_LOT	availabilityInLot	Count free slots and searching drivers in the lot.
PARKING_LOT	bestSectionInLot	Find the section with the most free slots.
PARKING_LOT	predictAvailability	ML-based prediction of future free slots.
PARKING_SECTION	availabilityInSection	Count free and total slots in the section.
PARKING_SLOT	checkSlotAvailability	Get the detailed status of a single slot.
PARKING_SLOT	updateSlotState	Change slot status (FREE/OCCUPIED) and refresh health.
VEHICLE	vehicleEnters / vehicleExits / reportSession	Record a vehicle event (PARKED / LEAVING / session state).

7. REST API, Roles & Authentication (Sprint 4)

Updated operations are invoked with API-Version: 1.4. Every operation (except account creation, update and login) authenticates the caller with userSystemID, userEmail and userPassword request parameters; creation and command invocation carry userPassword and take the identity from the request body (createdBy / invokedBy).

7.1 Role Permission Matrix

Role	Permitted operations
ADMIN	Only the Admin API: export all users, export command history, and delete-all (users / objects / commands). Object reads and writes are refused (HTTP 403).
OPERATOR	Create/update objects and bind relations (only on lots they own); read and search all objects, active or not, with pagination.
END_USER	Read and search active objects only (an inactive object is reported as 404, as if absent); the only role permitted to invoke commands.

7.2 Custom Search Queries

- GET /objects/search/byAlias/{alias} — exact-alias match.
- GET /objects/search/byAliasPattern/{pattern} — substring match on the alias.
- GET /objects/search/byType/{type} — all objects of a type.
- GET /objects/search/byStatus/{status} — all objects of a status.
- GET /objects/search/byLocation/{lat}/{lng}/{distance}?units=NEUTRAL|KM|MILES&shape=SQUARE|CIRCLE — objects within a square (default) or, as a bonus, a circle around the point.

All searches and list operations accept &size={size}&page={page} for pagination.

7.3 Example Requests

Search all parking lots (as an operator), first page of 20:

GET /ambient-invisible-intelligence/objects/search/byType/PARKING_LOT?userSystemID=2026b.Omer.Asayag&userEmail=operator@demo.org&userPassword=Oper@1&size=20&page=0

Invoke a command (as a driver):

POST /ambient-invisible-intelligence/commands?userPassword=User@1

{ "command": "availabilityInLot", "targetObject": { "id": { "objectId": "<LOT_ID>", "systemID": "2026b.Omer.Asayag" } }, "invokedBy": { "userId": { "email": "user@demo.org", "systemID": "2026b.Omer.Asayag" } }, "commandAttributes": {} }

8. System Architecture & Deployment

The system is delivered as three Git repositories:

- 2026b.omer.asayag — the Spring Boot API (main project): controllers, business logic, JPA data access, PostgreSQL.
- 2026b.omer.asayag_client — the two web/mobile clients (React + Vite): the driver mobile app (wrapped for Android with Capacitor) and the desktop staff console.
- 2026b.omer.asayag_predictive_service — the Python (Flask + XGBoost) ML microservice.

At runtime the browser/app calls the API over REST; the API calls the ML microservice for predictions; PostgreSQL (started via Docker Compose) provides persistence. In development the schema is recreated on each boot; for production a Spring profile keeps the data across restarts.

Ambient Invisible Intelligence - Spring 2026

Technologies Appendix

Smart Parking Management System - Spot&Slot

System ID: 2026b.Omer.Asayag · Sprint 4 (Final) · July 2026

1. Overview

This appendix lists the technologies used across the three repositories that make up the Spot&Slot Smart Parking system: the Spring Boot API (main project), the React clients (driver mobile app + staff console), and the Python machine-learning microservice.

2. Server — Smart Parking API

Technology	Version	Purpose
Java (JDK)	25	Implementation language (toolchain downloaded by the Gradle wrapper).
Spring Boot	4.1	Application framework: REST controllers, dependency injection, configuration, profiles.
Spring Web MVC	6.x	RESTful API with API versioning (API-Version header, v1.4).
Spring Data JPA / Hibernate	6.x	Object-relational persistence layer (repositories, paging, derived queries).
PostgreSQL	latest	Relational database (run in Docker). H2 in-memory is used as a dev/test fallback.
Spring Boot Docker Compose	4.1	Starts the PostgreSQL container automatically at boot from compose.yaml.
Spring RestClient	6.x	Calls the Python ML microservice for predictions.
springdoc-openapi (Swagger UI)	3.0	Interactive API documentation at /swagger-ui.html.
Gradle (wrapper)	9.3	Build tool; the wrapper pins Gradle and the JDK toolchain.

3. Server — Testing

Technology	Version	Purpose
JUnit 5 (Jupiter)	5.x	Test framework for the integration suite.
Spring MockMvc	6.x	Black-box testing of the REST layer.
Spring HTTP Interface clients	6.x	Declarative HTTP clients that call the running server over real HTTP.
Testcontainers	1.x	Spins up a throwaway PostgreSQL container for the tests.
Cucumber + Gherkin	7.23	Behaviour-driven tests: feature files + step definitions (cucumber-java, cucumber-spring, junit-platform-engine).
AssertJ	3.x	Fluent assertions (incl. recursive object comparison).

4. Clients — Mobile App & Staff Console

Technology	Version	Purpose
Node.js / npm	18+ / 22 LTS	Toolchain for building the web clients.
TypeScript	5.x	Typed implementation language for both apps.
React	18	UI library for the driver app and the staff console.
Vite	5	Build tool / dev server (two configs: mobile :8080, admin :8090).
React Router	6	Client-side routing and role-based route guards.
TanStack Query	5	Server-state data fetching, caching and polling.
shadcn/ui + Radix UI	—	Accessible UI component library.
Tailwind CSS	3	Utility-first styling.
Capacitor	8	Wraps the mobile web app into a native Android application (APK).
Recharts	2	Charts for the ML availability forecast.
Vitest + happy-dom	2	Unit tests for the shared client logic.
Android SDK / Gradle (AGP)	—	Builds the Android APK from the Capacitor project.

5. MachineLearning Microservice

Technology	Version	Purpose
Python	3.10+	Implementation language for the prediction service.
Flask	3.x	HTTP server exposing /predict and /forecast.
XGBoost	2.x	Gradient-boosted model (quantile bundle P10/P50/P90) for availability prediction.
scikit-learn	1.4+	Model evaluation and comparison during research.
pandas / NumPy	2.x	Data handling and feature engineering.
Joblib	1.3+	Serialises the trained model bundle (parking_model.pkl).
Holidays	0.50+	Israeli public-holiday calendar feature.
Jupyter Notebook	—	Model research and evaluation (model_research.ipynb).

6. Infrastructure & Tooling

Technology	Purpose
Docker / Docker Compose	Runs PostgreSQL for local development and the Testcontainers-based test suite.
Git / Bitbucket	Three separate repositories (API, clients, ML service), managed with feature branches and no-fast-forward merges.
REST / JSON	The contract between clients and server (base path /ambient-invisible-intelligence, versioned via the API-Version header).

Afeka College of Engineering

Ambient Invisible Intelligence - Spring 2026

Software Test Report

Smart Parking Management System - Spot&Slot

System ID: 2026b.Omer.Asayag · Base URL: http://localhost:8084/ambient-invisible-intelligence

Technology: Spring Boot 4.1 · JDK 25 · PostgreSQL (Spring Data JPA) · Docker

API Version: 1.4 · Sprint 4 (Final) · Date: July 2026

1. Test Approach

Behaviour is verified by an automated test suite that runs against a Testcontainers-managed PostgreSQL database, at three levels:

- JUnit integration tests over Spring Boot MockMvc (black-box, exercising the REST layer end to end).
- Declarative HTTP-interface client tests that call the running server over real HTTP.
- Cucumber / Gherkin BDD scenarios (Given/When/Then feature files) covering user accounts, per-operator lot management, parking commands and object search.

The full suite comprises 111 automated tests, all passing. The scenarios below document representative behaviour; each maps to one or more automated tests.

2. Roles & Credentials Used in Tests

Sprint 4 authenticates every non-account operation with userSystemID / userEmail / userPassword (creation and commands carry userPassword; identity comes from createdBy / invokedBy). Unless stated otherwise the scenarios use:

- admin@test.org / Test@1 (ADMIN) — the Admin API only.
- operator@test.org / Test@2 (OPERATOR) — builds and owns the parking objects; reads everything.
- enduser@test.org / Test@3 (END_USER) — invokes commands; reads active objects only.

All requests send API-Version: 1.4. Commands are POSTed to /ambient-invisible-intelligence/commands.

3. Test Scenarios

ID	Scenario	GIVEN	WHEN	THEN	Result
T-01	Lot availability counts	Seed users admin@test.org (ADMIN), operator@test.org (OPERATOR), enduser@test.org (END_USER). As the operator: create PARKING_LOT (status OPEN); sections A, B; slots A-1 (REGULAR,FREE), A-2 (REGULAR,OCCUPIED), B-1 (HANDICAP,FREE), B-2 (REGULAR,FREE).	END_USER POSTs command=availabilityInLot on the lot (userPassword)	200; freeSlots=3, searchingDrivers=0, worthEntering=true	PASS
T-02	Lot availability with HANDICAP filter	Same as T-01	availabilityInLot with slotKindFilter=HANDICAP	200; freeSlots=1	PASS
T-03	Best section in lot	Same as T-01 (A:1 free, B:2 free)	bestSectionInLot on the lot	200; bestSectionId=<B>, freeSlots=2	PASS
T-04	Predict availability	Same as T-01	predictAvailability with targetTimestamp	200; predictedFreeSlots>=0; note present	PASS
T-05	Predict falls back when ML down	Same as T-01; ML service stopped	predictAvailability	200; result = current free slots; fallback note	PASS
T-06	Section availability	Same as T-01	availabilityInSection on Section A	200; freeSlots=1, totalSlots=2	PASS

ID	Scenario	GIVEN	WHEN	THEN	Result
T-07	Slot availability (FREE)	Same as T-01	checkSlotAvailability on A-1	200; status=FREE, isFree=true, slotKind=REGULAR	PASS
T-08	Slot availability (OCCUPIED)	Same as T-01	checkSlotAvailability on A-2	200; status=OCCUPIED, isFree=false	PASS
T-09	Update slot state	Same as T-01; A-1 FREE	updateSlotState newState=OCCUPIED on A-1	200; later checkSlotAvailability A-1 = OCCUPIED	PASS
T-10	Update slot missing newState	Same as T-01	updateSlotState with no newState	200; result reports missing newState	PASS
T-11	Vehicle enters (PARKED)	Operator creates a VEHICLE (SEARCHING)	vehicleEnters on the vehicle	200; type=VEHICLE, status=PARKED	PASS
T-12	Vehicle exits (LEAVING)	Same as T-11	vehicleExits on the vehicle	200; status=LEAVING	PASS
T-13	Report session	Same as T-11	reportSession state=SEARCHING	200; type=VEHICLE, status=SEARCHING	PASS
T-14	Command-type validation (lot cmd on section)	Same as T-01	availabilityInLot on Section A	200; error 'not defined for object type PARKING_SECTION'	PASS

ID	Scenario	GIVEN	WHEN	THEN	Result
T-15	Command-type validation (section cmd on lot)	Same as T-01	availabilityInSection on the lot	200; error 'not defined for PARKING_LOT'	PASS
T-16	Command-type validation (vehicle cmd on slot)	Same as T-01	vehicleEnters on slot A-1	200; error 'not defined for PARKING_SLOT'	PASS
T-17	Missing invokedBy	Same as T-01	POST /commands with invokedBy null	400; JSON error body	PASS
T-18	Missing target systemID	Same as T-01	POST /commands, target systemID null	400; JSON error body	PASS
T-19	Blank command string	Same as T-01	POST /commands with command=""	400; JSON error body	PASS
T-20	ADMIN may not read objects	Same as T-01	GET /objects as admin@test.org	403 (Forbidden)	PASS
T-21	OPERATOR reads all objects	Same as T-01	GET /objects as operator@test.org	200; array length 7	PASS
T-22	END_USER reads active objects	Same as T-01	GET /objects as enduser@test.org	200; only active objects	PASS
T-23	OPERATOR may not invoke commands	Same as T-01	availabilityInLot invoked by operator@test.org	403 (Forbidden)	PASS

ID	Scenario	GIVEN	WHEN	THEN	Result
T-24	END_USER may not create objects	Same as T-01	POST /objects createdBy enduser@test.org	403 (Forbidden)	PASS
T-25	Non-owner OPERATOR cannot edit a lot	Same as T-01; second operator exists	PUT the lot as other-operator	403 (Forbidden)	PASS
T-26	Non-owner OPERATOR cannot delete a lot	Same as T-01; second operator exists	DELETE the lot as other-operator	403 (Forbidden)	PASS
T-27	OPERATOR exports users forbidden	Same as T-01	GET /admin/users as operator	403 (Forbidden)	PASS
T-28	END_USER cannot see an inactive object	Same as T-01; operator sets A-1 active=false	GET /objects/{sys}/A-1 as enduser	404 (as if absent); operator still gets 200	PASS
T-29	END_USER search excludes inactive	Same as T-28 (B-2 set inactive)	search byStatus/FREE as enduser vs operator	enduser: 2 results; operator: 3 results	PASS
T-30	Command on inactive target rejected	Same as T-01; lot set inactive	availabilityInLot on the inactive lot	404 (target treated as absent)	PASS
T-31	Search by exact alias	Same as T-01	GET /objects/search/byAlias/A-1	200; 1 result, alias=A-1	PASS
T-32	Search by alias pattern	Same as T-01	GET /objects/search/byAliasPattern/A-	200; 2 results (A-1, A-2)	PASS

ID	Scenario	GIVEN	WHEN	THEN	Result
T-33	Search by type	Same as T-01	GET /objects/search/byType/PARKING_SLOT	200; 4 results	PASS
T-34	Search by status	Same as T-01	GET /objects/search/byStatus/FREE	200; 3 results	PASS
T-35	Search by location (square)	Two lots at (32,34) and (33,35)	byLocation/32.0/34.0/0.5	200; 1 result (near lot)	PASS
T-36	Location units=KM conversion	Same as T-35	byLocation/32.0/34.0/60?units=KM	200; 1 result (~0.54°)	PASS
T-37	Location shape=CIRCLE (bonus)	Center + corner lot	byLocation .../0.5?shape=CIRCLE	200; 1 result (corner excluded)	PASS
T-38	Location invalid units	Same as T-35	byLocation .../1?units=PARSEC	400 (Bad Request)	PASS
T-39	Pagination slices deterministically	7 seeded objects	GET /objects?size=3&page=0..3	lengths 3, 3, 1, 0	PASS
T-40	Pagination rejects size=0	Same as T-01	GET /objects?size=0	400 (Bad Request)	PASS
T-41	Read requires valid password	Same as T-01	GET /objects with wrong password	401 (Unauthorized)	PASS
T-42	Read requires credentials	Same as T-01	GET /objects with no credentials	401 (Unauthorized)	PASS

ID	Scenario	GIVEN	WHEN	THEN	Result
T-43	Login succeeds	admin@test.org / Test@1 exists	GET /users/login/{sys}/admin@test.org?password=Test@1	200; role=ADMIN	PASS
T-44	Login wrong password	admin@test.org exists	login with password=wrong	401 (Unauthorized)	PASS
T-45	Create user without username	Server up	POST /users with no username	400 (Bad Request)	PASS
T-46	Create user with blank avatar	Server up	POST /users avatar=""	400 (Bad Request)	PASS
T-47	Update user to blank username	admin@test.org exists	PUT /users username=""	400 (Bad Request)	PASS
T-48	Create object with blank status	Operator authenticated	POST /objects status=""	400 (Bad Request)	PASS
T-49	Create object without type	Operator authenticated	POST /objects, no type	400 (Bad Request)	PASS
T-50	Create user with weak password	Server up	POST /users password=abc	400 (Bad Request)	PASS
T-51	Create ADMIN needs acting admin	Server up	POST /users role=ADMIN, no actingEmail	401/403 (rejected)	PASS

ID	Scenario	GIVEN	WHEN	THEN	Result
T-52	Create slot returns details	Same as T-01	operator POSTs PARKING_SLOT A-3	200; parentLotId, slotKind correct	PASS
T-53	Delete lot cascades	Same as T-01	operator DELETE the lot	200; lot, sections and slots all 404	PASS
T-54	Delete section cascades to its slots	Same as T-01	operator DELETE Section A	200; A-slots 404; B-slots and lot remain	PASS
T-55	Bind child and read children	Same as T-01	PUT /{lot}/children (section); GET children	200; children include the section	PASS
T-56	DELETE /admin/objects clears all	Same as T-01	admin DELETE /admin/objects; operator GET /objects	200; then empty array	PASS
T-57	Get non-existent object	Empty DB	operator GET /objects/{sys}/missing	404; JSON error body	PASS
T-58	Admin exports command history	A command was invoked	GET /admin/commands as admin (paginated)	200; array of command records	PASS
T-59	CORS preflight allows app origins	Server up	OPTIONS /objects, Origin capacitor://localhost	200; Access-Control-Allow-Origin echoed	PASS
T-60	CORS preflight refuses unknown origin	Server up	OPTIONS /objects, Origin https://evil.example.com	403 (Forbidden)	PASS

Total: 60 documented scenarios, all passing. These behaviours are backed by an automated suite of 111 tests (JUnit MockMvc + declarative HTTP-interface clients + Cucumber BDD) that run against a Testcontainers-managed PostgreSQL; all 111 passes.

4. Performance Testing (Apache JMeter)

A load test was executed with Apache JMeter 5.6.3 against the running API to measure latency and throughput under concurrency. The test plan (perf/smart-parking-perf.jmx in the API repository) sends the API-Version: 1.4 header, authenticates with the seeded roles (operator@demo.org / OPERATOR and user@demo.org / END_USER), resolves a real PARKING_LOT id in a setUp phase, then exercises paged reads, the custom search queries and a domain command. Every sample is asserted to return HTTP 200.

Configuration: 10 concurrent virtual users, 20 iterations each (~1,000 requests), 100-400 ms think time between requests. Server: Spring Boot 4.1 on an in-memory database, run locally. These figures are a functional baseline measured on a developer machine with the load generator on the same host; a production PostgreSQL deployment on separate hardware will differ.

Request	#	Avg	p90	p95	Max	req/s	Err
Total	1001	23.5	54	75	234	29.6	0
R1 GET /objects (paged)	200	18.0	34	47	97	6.4	0
R2 search/byType PARKING_SLOT	200	18.7	38	46	138	6.3	0
R3 search/byStatus FREE (active-only)	200	17.6	35	49	90	6.4	0
R4 search/byLocation 5 KM	200	8.4	16	21	57	6.4	0
C1 command availabilityInLot	200	54.2	90	104	234	6.4	0

Response times in milliseconds; p90/p95 are the 90th/95th percentiles.

Result: 1,001 requests, 0.00% errors. Reads and searches respond in single-digit to ~19 ms on average (the geographic search is cheapest at ~8 ms); the availabilityInLot command is the heaviest at ~54 ms average and ~104 ms at the 95th percentile, as expected since it scans a lot's slots and counts searching vehicles rather than performing a single indexed lookup. All operations stayed well within interactive latency under the tested concurrency.

AFEKA
The Academic College of Engineering in Tel Aviv

Afeka College of Engineering

Department of Software Engineering

Ambient Invisible Intelligence - Spring 2026

Spot&Slot - UX/UI Research

Smart Parking Management System

SPOT&SLOT

"The Moovit of Parking"

System ID: 2026b.Omer.Asayag | Date: July 2026

1. Summary of Research Findings - Target Audience Characteristics

After our initial research, we expanded our research from the college of Afeka to the entire subsociety of north Tel Aviv. We focused and adapted the recent research in target audience in the north part of Tel Aviv.

1) Residents

- The largest and most affected group. North Tel Aviv residents, spanning the Old North (Tzafon Yashan), New North (Tzafon Hadash), and parts of Ramat Aviv, face a chronic, daily parking struggle. Many residential buildings in these neighbourhoods were built before mandatory parking requirements, leaving residents without dedicated garage spaces. Studies on urban parking stress (Shoup, 2005) show that in high-density neighbourhoods with insufficient off-street supply, residents spend an average of 15-20 minutes per day searching for curb parking, contributing to roughly 30% of local traffic congestion. This group includes young professionals (25-40), established families, and longtime residents (50+), all united by the same nightly frustration of returning home to find no parking on their street.

- Residents exhibit high smartphone literacy, especially the younger segment, and are the most motivated adopters of a parking solution because they face the

problem every single day. Their primary pain window is the evening return (18:00–21:00), when street parking is at maximum occupancy due to overlapping resident returns and restaurant traffic.

- **A key behavioural insight:** once a resident finds a spot, many avoid moving their car for the rest of the evening (or even the weekend) for fear of losing it. This "parking hoarding" behaviour further reduces turnover and deepens the scarcity cycle.

2) Workers, Students & Local Businesspeople

- Professionals who commute to offices, clinics, and businesses in North Tel Aviv. This includes employees at tech companies on Weizmann and Raoul Wallenberg streets, medical staff at clinics, accountants, lawyers, and shop owners along Ibn Gabirol and Dizengoff. Also, students which shared the same pattern of routine. They arrive during morning hours (08:00-10:00) and occupy spots throughout the business day. Their primary need is predictable, hassle-free parking near their workplace, and they are often willing to walk a few blocks if they can be certain a spot exists.
- This group is generally tech-comfortable and open to app-based solutions, particularly those that integrate with their existing morning routine (checking availability before leaving home). They create a significant daytime demand layer that overlaps with short-term visitor parking, especially near commercial streets.
- Unlike residents, workers typically have more flexibility, they can adjust departure time by 15–30 minutes to catch a better availability window, making predictive forecasting especially valuable for this segment.

3) Dining, Shopping & Entertainment Visitors

- A large and diverse population arriving from across the Tel Aviv metro area for the vibrant restaurant scene on Ibn Gabirol, Dizengoff, and Basel Streets, boutique shopping, and cultural events at Habima Theatre, the Cameri, and Mann Auditorium. This group arrives primarily in the afternoon and evening hours, peaking on Thursday nights and weekends. They are typically unfamiliar with the

neighborhood's one-way street grid, resident-only zones, and hidden parking pockets. Their tolerance for circling is low — after 5–10 minutes of frustration, many resort to expensive commercial lots (Dizengoff Center, Basel parking) or risk parking in prohibited zones. They are highly receptive to a real-time map showing available spots near their specific destination.

4) Beachgoers & Weekend Leisure Seekers

- A seasonal but high-impact group that creates massive demand spikes on warm weekends and holidays. They head to Hilton Beach, Nordau Beach, and Gordon Beach, arriving in waves from late morning through afternoon (10:00–15:00). Many arrive from outside Tel Aviv and have minimal familiarity with the local street layout. This group competes directly with residents for the same curb spaces on streets like Ben Yehuda, Nordau, and HaYarkon. Their parking needs are short-to-medium duration (2–4 hours), and they are often willing to walk further if guided to an available spot. Weekend beach traffic is one of the most predictable demand patterns, making it ideal for forecast-based guidance.

Target Audience Characteristics Summary

The North Tel Aviv parking audience spans four main groups: residents facing chronic daily shortages, workers needing reliable daytime spots, dining and entertainment visitors creating evening and weekend spikes, and beachgoers generating seasonal surges. Each group has distinct time patterns, familiarity levels, and willingness to walk or pay — but all share the same core need: real-time certainty about where a legal, available spot exists.

1. Product Goal - Addressing the Following Problems:

While our initial research on the academic campus identified pain points primarily related to tardiness and parking in aisles, expanding to the dense urban environment of North Tel Aviv reveals far more acute and dangerous behaviours, such as double-parking and blocking pedestrian pathways.

- **The Core Problem - Safety:** Parking shortages on North Tel Aviv streets lead to reckless double-parking on Ibn Gabirol, blocking bike lanes on Arlozorov, and parking on sidewalks in residential side streets. This creates risks for pedestrians, cyclists, and drivers, blocks lines of sight at intersections, and hinders emergency vehicle access.

- **Tardiness to Appointments & Events:** Prolonged searches for parking cause late arrivals to work meetings, medical appointments, restaurant reservations, and cultural events, disrupting personal and professional schedules.

- **Excessive Time Waste:** Uncertainty about parking availability forces residents and visitors to leave earlier than necessary, adding 15–30 minutes of unproductive circling to every car trip, and negatively impacting quality of life.

- **Fines & Enforcement Anxiety:** Parking distress often leads to parking in prohibited zones, loading areas, or resident-only spaces, resulting in unexpected municipal fines (starting at ₪250) and towing.

- **Stress & Frustration:** The daily race to find parking, especially in the evenings when residents return home and streets are packed with diners, creates chronic stress that degrades the urban living experience and contributes to aggressive driving behaviour.

3. Product Usage Environment

We decided to split the usage environments to better understand the needs and contexts of use for parking in the North Tel Aviv urban area.

Comparison Criterion	Environment A: Home (pre-planning)	Environment B: Morning Prep (Heading out)	Environment C: In the Car (Active search)
Physical Description	Static, relaxed environment, stable lighting at home or office.	Dynamic environment, user in motion at home or walking to the car on the street.	Noisy environment, changing lighting, crowded narrow North Tel Aviv streets with cyclists and pedestrians.
Mental State & Motivation	Low stress. Motivated to gain certainty and plan the best time to drive / return home.	Moderate to high stress due to time pressure. Motivated to make a quick tactical decision.	Maximum stress. Critical "pain point," motivated for an immediate solution to avoid circling and double-parking.

Comparison Criterion	Environment A: Home (pre-planning)	Environment B: Morning Prep (Heading out)	Environment C: In the Car (Active search)
Technical Context & Devices	Use of mobile or desktop devices, browsing via stable internet connection.	Mobile device only (usually one-handed), switching to cellular network.	Mobile device mounted on a dashboard or car interface, critical reliance on GPS.
Main Tasks (Task Analysis)	Checking statistical forecasts and availability predictions for the destination zone/home neighbourhood.	Quick comparison between the forecast and real-time zone status.	Precise navigation to a vacant street parking spot and receiving voice updates.
Usage Constraints & Barriers	Almost no constraints, the user is free to delve into information.	Cognitive load due to multi-tasking.	Severe safety constraints. Limited screen interaction and taking eyes off the road.
Interface Requirements (UI/UX)	Information-rich display, charts, and detailed data.	Use of colors and icons for rapid messaging.	"Touchless" interface, voice feedback, large buttons, and high contrast.

4. Survey Execution and Insights

Part A: General Characteristics

1. **What is your connection to the North Tel Aviv area?**
    - Resident
    - Works in the area
    - Visitor / Leisure
2. **How often do you drive to the North Tel Aviv area?**
    - Every day
    - 3-4 times a week
    - Once or twice a week
    - Rarely / do not drive there
3. **At what hours do you usually arrive in the area?**
    - Morning (07:30 - 10:00)

- Midday (10:00 - 14:00)
- Afternoon / Evening (14:00 - 20:00)
- Night (20:00 onwards)

Part B: Parking Habits and Behaviour

4. **On average, how much time do you spend searching for parking in North Tel Aviv?**
   - 0-3 minutes (find it immediately)
   - 3-10 minutes
   - 10-20 minutes
   - Over 20 minutes
   - Sometimes give up and park very far away or in a prohibited zone
5. **How often do you encounter difficulty finding parking?**
   - Every time I come
   - 2-3 times a week
   - Occasionally
   - Never
6. **What is your primary tactic for finding parking today?**
   - Driving in circles in the neighbourhood until someone leaves
   - Following pedestrians walking to their cars
   - Driving directly to a paid parking lot (Dizengoff Centre, etc.)
   - Parking illegally (double-parking, on sidewalks, in loading zones)
   - Other

Part C: The Parking Experience (Table)

Statement	1 (Strongly Disagree)	2	3	4	5 (Strongly Agree)
Searching for parking near my home/destination in North TLV causes me stress	O	O	O	O	O

Statement	1 (Strongly Disagree)	2	3	4	5 (Strongly Agree)
I have been late to an appointment or event solely because I couldn't find parking	O	O	O	O	O
I avoid moving my car once parked for fear of losing my spot	O	O	O	O	O
I tend to leave early just to find a vacant spot near my destination	O	O	O	O	O
I feel that driving through the narrow streets of North TLV while searching	O	O	O	O	O

Statement	1 (Strongly Disagree)	2	3	4	5 (Strongly Agree)
for parking is unsafe					

Part D: Needs and Preferences

7. **If there was an app showing real-time street parking status in North Tel Aviv, what would be most important to you?**
    - A graphic map display showing exactly where a green (vacant) spot is
    - Voice navigation that automatically directs me to the closest vacant spot to my destination
    - Statistical information before leaving home (Are there spots available near my destination right now?)
    - Ability to save where I parked and set a timer for the parking meter
    - Push notifications when a spot opens near my saved destinations
8. **Do you have any additional comments regarding the parking experience in North Tel Aviv? (Optional open-ended text)**

Key Survey Insights:

The survey was submitted by 69 people. 30 of them were residents of the area, 15 of them were workers, students & businesspeople, 15 of them were Dining, Shopping & Entertainment Visitors, 9 of them were Beachgoers & Weekend Leisure Seekers.

1. **The core problem is uncertainty:** Even after expanding the target audience, the result stays similar. About 70% report struggling to find parking at least twice a week in North Tel Aviv, and ~25% face it daily. ~85% spend 5-15 minutes circling before finding a spot. Insight: The issue isn't just physical space but inefficient searching. Users "cruise" (drive in circles), contributing to an estimated 30% of traffic congestion in the area (consistent with Shoup's cruising estimates for dense urban zones).

2. **High mental impact:** A significant percentage reported chronic stress related to parking, particularly residents returning home in the evening and visitors arriving for dinner reservations. Insight: The system must provide "peace of mind," not just a location. The

interface must be calming and clear to reduce cognitive load while driving through the narrow, one-way streets of the Old North.

3. **Dangerous behaviour and illegal parking:** Out of desperation, ~30% admit to double-parking "just for a minute," blocking bike lanes on Arlozorov, or parking in loading zones. Insight: The current situation pushes law-abiding citizens into hazardous violations. The solution must provide certainty regarding legal vacant parking to prevent dangerous improvisations that endanger cyclists and pedestrians.

4. **Gap in needs between residents and visitors:** Residents seek parking near their home and dread the evening return; visitors need proximity to a specific destination (restaurant, shop, beach) and are more open to paid solutions.

5. **Preference for real-time visual solutions:** An overwhelming preference for a visual street-map displaying "green/red" availability in real-time. Insight: A functional requirement for a map-based interface overlaid on actual North Tel Aviv streets with minimal text.

Summary of Insights:

The survey confirms the urgent need for a technological parking management solution for the North Tel Aviv area. Users seek certainty and time savings, with minimal phone interaction while navigating the busy urban grid.

5. Persona Definition and User Audience

Based on the target audience and survey, we defined several personas to understand our app's target market in the North Tel Aviv area:

Persona: Noa - The Young Professional

- **Gender & Age:** Female (30).
- **Profession:** UX Designer at a tech startup in Ramat HaHayal.
- **Status:** In a relationship, rents a 2BR apartment on Jabotinsky St, Old North.
- **Education:** B.Des in Visual Communication.
- **Tech Experience:** Very high ("Digital Native"). Manages all daily needs via smartphone (Wolt, Gett, Google Maps, Cibus).
- **Traits:** Time-poor, impatient with inefficiency, design-conscious, values her evening routine, frustrated by parking near her apartment building that has no garage.

- **Habits:** Drives to work and back daily. Returns home around 19:00-20:00 when streets are packed. Circles her block 3-4 times before finding a spot. Sometimes parks 10-minute walk away and carries groceries.
- **Needs:** Real-time street availability near her home address, notification when a spot opens on her street, accurate prediction of when parking eases up in the evening.
- **Motivations:** Eliminate the nightly parking frustration, reduce commute stress, reclaim 15-20 minutes of her evening.
- **Usage Context:** In the car returning from work, ~10 mins from home, tired and hungry, navigating narrow one-way streets in the Old North.

Persona: Rami - The Longtime Resident & Family Man

- **Gender & Age:** Male (58).
- **Profession:** Senior Accountant, self-employed, office on Weizmann St.
- **Status:** Married + 2 (adult children), owns an apartment on Nordau Blvd since 1998.
- **Education:** B.A. in Economics and Accounting.
- **Tech Experience:** Medium. Uses Waze for navigation, WhatsApp, and basic smartphone functions. Dislikes complex apps and tiny fonts.
- **Traits:** Established, routine-oriented, protective of "his" parking spot near home, increasingly frustrated by the neighbourhood's densification and nightlife traffic. Mild vision impairment (needs clear fonts and high contrast).
- **Habits:** Drives to his office 5 minutes away every morning. Parks easily at 08:00 but dreads returning after 18:00. On weekends, avoids moving his car entirely to avoid losing his spot. Has a mental map of "secret" spots but they've become unreliable.

- **Needs:** Certainty about street availability near Nordau Blvd before heading home, minimal physical walking distance (carries briefcase and sometimes groceries), clean UI with large text and high contrast.
- **Motivations:** Protect his daily routine, avoid the rage of circling his own neighbourhood, maintain quality of life in a rapidly changing area.
- **Usage Context:** In the car, heading home from his office, deciding whether to go straight home or stop for errands first depending on parking availability.

Persona: Tomer - The Weekend Visitor & Foodie

- **Gender & Age:** Male (35).
- **Profession:** Product Manager at a fintech company. Lives in Givatayim.
- **Status:** Married + 1 toddler, drives to North Tel Aviv 2-3 times a week for restaurants, friends, and beach outings.
- **Education:** MBA.
- **Tech Experience:** Very high. Heavy user of restaurant reservation apps (Tabit, Google Maps), navigation, and payment apps. Expects instant system responsiveness.

- **Traits:** Social, spontaneous, hates wasting time on logistics, geographically familiar with the area but has no intuition for when/where spots open.
- **Habits:** Drives to North TLV for Friday lunch, Thursday night dinners, and weekend beach trips. Has been late to reservations multiple times. Once circled for 25 minutes near Basel St. Has resorted to expensive Dizengoff Center parking out of desperation.
- **Needs:** Quick, confident decision (street park or pay for a lot?), proximity to his restaurant/destination, avoidance of fines in unfamiliar resident-only zones.
- **Motivations:** Arrive at his dinner reservation on time and relaxed, spend quality time with family instead of circling, avoid the ₪250 parking ticket.
- **Usage Context:** Driving from Givatayim on a Thursday evening, 15 minutes from the restaurant, with his wife and toddler in the car, needing an immediate parking solution.

User Audience Breakdown

Primary Users:

Use the app frequently (daily or several times a week). Require fast, accurate data while driving through North Tel Aviv streets.

- Residents: The core demographic. Need parking near their homes every evening. Suffer chronic frustration with the nightly parking hunt and weekend spot hoarding.
- Workers & Businesspeople: Commute to offices, clinics, and shops in the area. Need predictable morning parking and benefit most from occupancy forecasting.
- Regular Dining & Shopping Visitors: Come 2–4 times a week for restaurants, cafés, and shopping. Know the area but lack real-time parking intuition.

Secondary Users: Occasional or seasonal visitors.

- Beachgoers: Arrive in waves on warm weekends, creating massive demand spikes near Hilton/Nordau/Gordon beaches.
- Cultural Event Attendees: Theatre, concerts at Mann Auditorium, gallery openings — need parking for specific evening time windows.
- Delivery Drivers & Service Providers: Need short-term parking near specific addresses.
- Tourists: Unfamiliar with the area entirely, would benefit from clear visual parking guidance.

6. Summary of Interview Findings

Noa (The Young Professional)

- **Motivation:** Reclaim her evening, reduce daily frustration, eliminate wasted time.
- **Goals:** Know parking availability near her apartment 10 mins before arriving, minimize circling, get real-time accuracy on her specific street.
- **Behaviour:** Checks phone at red lights, multi-tasks (driving/app), adopts new tech immediately if it works.
- **Knowledge:** Knows every street in the Old North by heart but has no real-time status. Highly skilled with standard apps.
- **Current UX & Pain Points:** Uses trial-and-error (drives home → circles block → gets frustrated → parks far away). Suffers from lack of transparency, wasted time, and emotional exhaustion after a long workday.

Rami (The Longtime Resident)

- **Motivation:** Preserve his quality of life, operational certainty about parking near his home, physical comfort.

- **Goals:** Know if there's a spot near Nordau Blvd before leaving the office, minimal UI interaction, no surprises upon arrival.
- **Behaviour:** Checks before leaving work, tech conservative, relies on clear visuals (large fonts, simple colour coding).
- **Knowledge:** 25+ years in the neighbourhood, knows every "secret" spot. Needs simple language ("Available" vs "Full").
- **Current UX & Pain Points:** Relies on instinct. If his usual spots are taken, he circles angrily and sometimes parks 3 blocks away, carrying heavy bags. Increasingly feels the neighbourhood is "not what it used to be."

Tomer (The Weekend Visitor)

- **Motivation:** Enjoy North TLV without logistics stress, maximize family time, avoid fines.
- **Goals:** Decide before arriving, street park or lot? Zero wasted circling, lower cognitive load with a toddler in the car.
- **Behaviour:** Checks apps at home before leaving, relies on dashboard-mounted phone, abandons unreliable tech instantly.
- **Knowledge:** High tech literacy knows the main streets but not the back-alley parking dynamics or resident-only zones.
- **Current UX & Pain Points:** Gambles on finding a spot. Has been late to reservations, gotten fines in resident-only zones he didn't notice, and paid ₪60+ for Dizengoff Centre parking out of desperation.

7. Future Use Cases (Scenarios)

- **Viewing Future Occupancy Forecast:** User opens app, selects a neighbourhood zone (e.g., "Basel St area") and a future date/time. The system calculates a forecast based on historical data and known events. Alternatives: Low confidence warning if insufficient data; event-aware predictions (e.g., a concert at Mann Auditorium will spike demand).
- **Checking Status While Driving:** User opens app a route to North Tel Aviv. Chooses "Check Parking Status" near a saved destination. The system analyses real-time data. The app displays zone level availability (Available/Tight/Full), guiding the user's decision on whether to try street parking or head to a paid lot. Alternatives: Offline mode shows last update; App recommends a nearby paid lot if the zone is full.
- **Viewing Real Time Snapshot:** User opens app. The system analyses municipal sensor data and Computer Vision feeds to determine available street spots. The app displays a mapped visual overlaid on the actual street grid. Users can refresh anytime. Alternatives: Temporary unavailability message if a sensor feed drops; Indicator if data is slightly delayed.

8. Competitor Analysis

Analysis of competitors for the Spot&Slot system in urban street parking:

Pumba

- **Description:** Social-tech network using physical street sensors for real-time parking reporting in select Tel Aviv streets.
- **Pros:** High accuracy (physical sensor), targeted notifications.
- **Cons:** Extremely high setup cost, complex hardware maintenance, very limited street coverage.
- **Usability:** High locally where deployed, but coverage gaps make it unreliable for whole neighbourhood use.

Cello (formerly Cellopark)

- **Description:** Israel's most popular parking payment app featuring a "Parking Hunter" based on statistical analysis of payment expirations.
- **Pros:** Low setup cost (no hardware), centralized services (pay + search), massive existing user base.
- **Cons:** Lacks absolute accuracy (statistical probability based on meter expirations, not visual confirmation), no neighbourhood level forecasting.
- **Usability:** Medium for parking hunting. Great for payments, but users still "cruise" and hope for the best.

Pink Park

- **Description:** Platform for renting out private parking spaces (apartment building garages, private lots) when vacant.
- **Pros:** Guaranteed pre-booking, asset utilization for building residents.
- **Cons:** Very limited inventory in North Tel Aviv (most buildings lack garages), requires building management cooperation, paid service.
- **Usability:** Low for spontaneous street parking needs. Only serves pre-planned trips.

Unlike the academic campus environment where solutions like Pink Park were largely irrelevant, in North Tel Aviv it serves as a partial alternative, though still heavily limited by the lack of private garages in older residential buildings.

Weighted Competitor Analysis Table

Criterion	Weight	Spot&Slot	Pumba	Cello	Pink Park
Real Time Accuracy & Reliability (CV)	35%	5 (Computer Vision)	4 (Sensors)	2 (Statistical)	4 (Gate mgmt)
Setup & Maintenance Cost	25%	4 (Existing cameras)	2 (Expensive hardware)	5 (App only)	3 (Gate hardware)
Occupancy Forecasting	20%	4 (Built-in forecast)	1 (Real-time only)	2 (Past data)	1 (Availability only)
Urban Street Suitability	20%	5 (Tailored for the North Tel Aviv region)	2 (Urban, but sparse deployment)	3 (General municipal)	2 (Private lots only)
Final Weighted Score	**100%**	**4.55**	**2.50**	**2.95**	**2.75**

Analysis Insights:

The survey was conducted among 50 participants, distributed between 25 North Tel Aviv residents, 15 regular visitors/workers, and 10 occasional visitors.

- **Technological Advantage:** Spot&Slot uses Computer Vision on existing municipal and commercial security cameras, allowing high accuracy at a low cost, unlike Pumba's

expensive per-spot sensors or Cello's inaccurate meter-expiration guesses. Pink Park focuses on private garage inventory, which barely exists in the pre-1970s building stock of the Old North.

- **Market Differentiation:** Spot&Slot turns street parking into an "insight." By providing future occupancy forecasts per neighbourhood zone and integrating event-awareness (concerts, holiday eves, weekend beach rushes), it enables proactive planning, a strategic advantage for both municipal traffic management and everyday residents and visitors.

8. Feedback & Refinement of The Interface

Important Note: The UX/UI design is an iterative process, but at this moment this is our best version.
The prototypes currently presented utilize the closed-lot interface ('Demo Lot') developed during our initial research. Based on the North Tel Aviv survey insights, which showed an overwhelming preference for a visual street map, the next iteration of the UI will be fully adapted to a street level urban map display.

Conceptual Characterization

- Visual display
- Minimalist display
- Soft and calming design
- Current street availability display
- Clear future occupancy forecast by neighbourhood zone

We have taken these notes in consideration, and we have implemented them in the first version of the prototype.

Initial Prototype:



Feedback

After we conducted a “semi” forced alpha testing on our product with residents and visitors in the North Tel Aviv area, the users explained verbally what was uncomfortable in their experience.

The Pain of the Users

The users had presented several “pain points”:

- A stressing UI (colours & interface)
- Unsuitable interface for Hebrew users (left-to-right, ltr)
- Uncleared parts in the menu (the menu was not universally understandable)
- Cluttered Interface

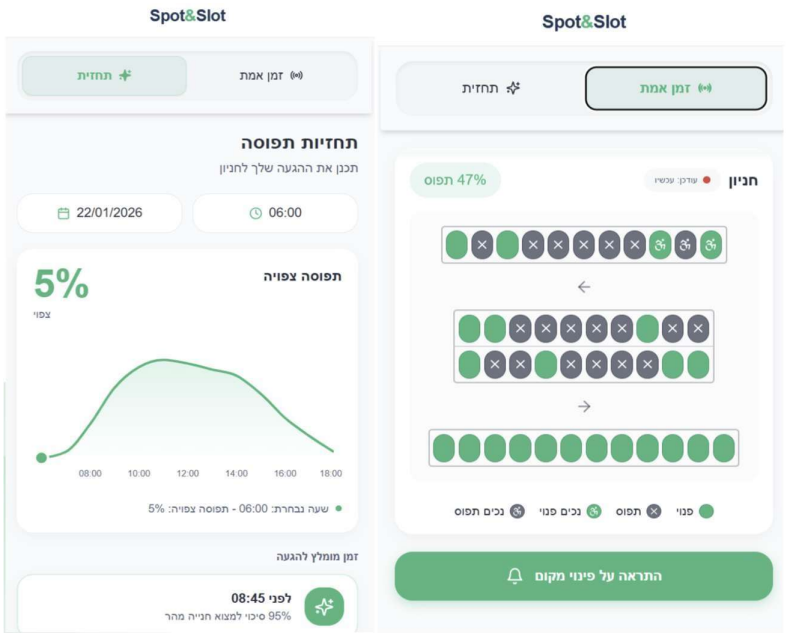
Design Improvements

Due to the “pain” of the users a few updates we made:

- Changed red to grey

- Moved headings to the right (or: Right-aligned headers)
- Increased font size
- Removed "Smart Insights"
- Moved "Alert" button to the main screen
- Changed timeline direction from left to right
- Moved "Occupancy Trend" to the secondary screen
- Added icons

Secondary Prototype:



9. Extended Functionality - UX Research & Rationale

As the product matured beyond the initial driver-facing prototype, the availability forecast was deepened, it now communicates not only a number but its confidence level. This capability was approached with the same user centred method used throughout this document: define the user and their context, then ground every interface decision in established UX principles.

9.1 Forecast & Uncertainty - Design Rationale

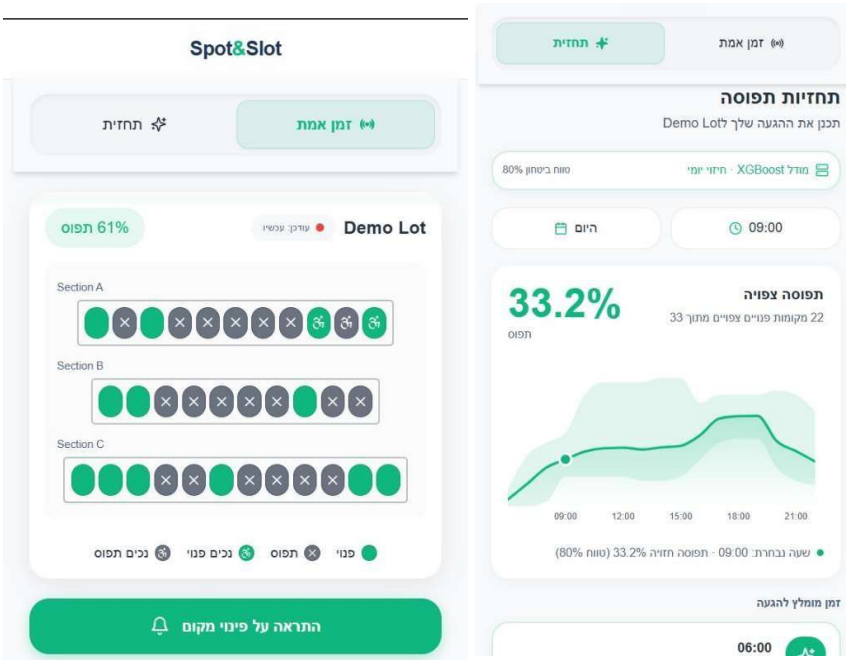
This work extends the 'Viewing Future Occupancy Forecast' scenario from Section 7 and the survey finding that users want 'statistical information before leaving home.' The early prototype reduced the forecast to a single figure, which created two UX problems: a lone number implies false precision (it tells the user the system is certain when it is not), and drivers plan around a window of time, not one instant.

Show the range, not just the point. The forecast now renders a median line inside a shaded P10–P90 confidence band. Visualizing prediction uncertainty is established practice for predictive interfaces (Padilla et al.): it sets honest expectations, builds appropriate rather than blind trust, and discourages over-reliance on an exact value. This serves the survey's clearest emotional finding, users want peace of mind and a calm, honest interface, not false promises.
A full day briefly, for planning. Instead of a single time, the user sees the whole day's curve and can scrub a date and time. This supports the 'strategic pre-planning' behaviour observed for Rami and Tomer ('check before leaving home', 'decide 15 minutes out'), and a 'recommended time to arrive' highlights the calmest window.

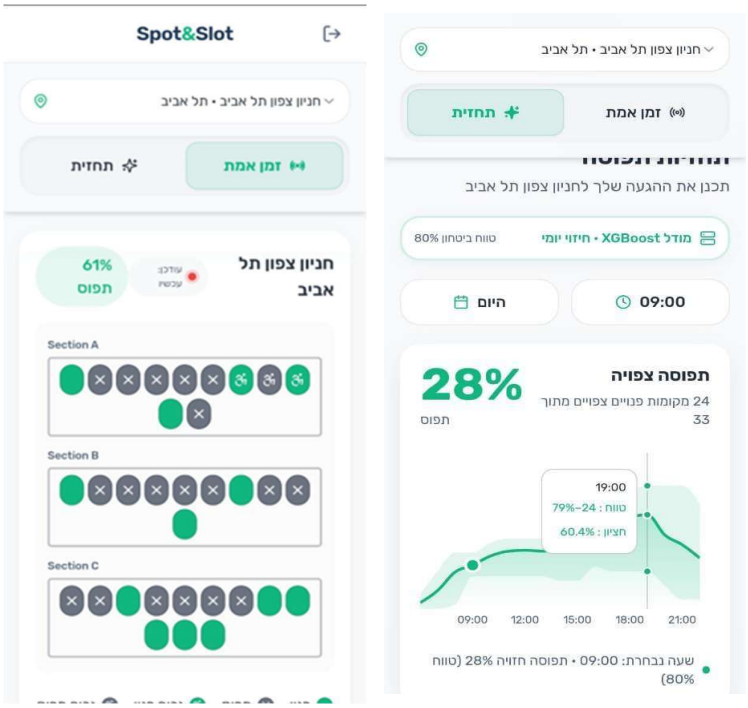
Honest about special days. Israeli public holidays, Shabbat transitions, and major events (White Night, Tel Aviv Marathon route closures) swing demand dramatically. The forecast flags these events and shows the correspondingly different occupancy pattern, so an unusual prediction reads as explained rather than broken, again, visibility of system status.
Consistent with the calm aesthetic. The band is a soft, translucent tint of the primary colour with no alarming reds, consistent with the Section 8 decisions to soften the palette ('changed red to grey') and keep the experience calming.

As with the rest of the interface, these additions follow the iterative principle stated in Section 8: they reflect our best current understanding of each user's context and remain open to refinement.

Prototype Before expanding the Target Audience:



Prototype After expanding the Target Audience:



Bibliography

- Shoup, D. (2005). The High Cost of Free Parking. American Planning Association.
- Pew Research Center (2023). Mobile Technology and Home Broadband 2023. Pew Research Center, Washington, D.C.
- Arnott, R., & Inci, E. (2006). An integrated model of downtown parking and traffic congestion. Journal of Urban Economics, 60(3), 418-442.
- Nielsen, J. (1994). 10 usability heuristics for user interface design. Nielsen Norman Group.
- Shneiderman, B. (1996). The eyes have it: A task by data type taxonomy for information visualizations. Proceedings of the IEEE Symposium on Visual Languages, 336-343.
- Few, S. (2006). Information dashboard design: The effective visual communication of data. O'Reilly Media.
- Ware, C. (2012). Information visualization: Perception for design (3rd ed.). Morgan Kaufmann.
- Padilla, L. M., Creem-Regehr, S. H., Hegarty, M., & Stefanucci, J. K. (2018). Decision making with visualizations: A cognitive framework across disciplines. Cognitive Research: Principles and Implications, 3(29).

Afeka College of Engineering

Department of Software Engineering

Ambient Invisible Intelligence - Spring 2026

Model Evaluation Report

Parking Availability Prediction — Smart Parking

"Spot&Slot" — The Moovit of Parking

Best model: XGBoost | $R^2 = 0.948$ | MAE = 23.56 free slots

System ID: 2026b.Omer.Asayag | Date: June 2026

Generated from model_research.ipynb

1. Introduction & Objective

The Smart Parking system predicts how many parking slots will be free in a lot at a future time, so drivers can plan ahead. This report documents the data-science study behind that predictor: how the training data was produced, which machine-learning models were compared, how they were evaluated, and which model is recommended for production. All results are reproducible from the accompanying Jupyter notebook (model_research.ipynb).

Target variable: free_slots (number of available parking slots). Best performing model: XGBoost — $R^2 = 0.948$, mean absolute error = 23.56 free slots (7.27% of average lot capacity).

2. Dataset

A synthetic but realistic hourly dataset was generated for five real North Tel Aviv parking lots over a full year (2024-09-01 .. 2025-08-31), giving 43,800 rows. Crucially, the lots have different capacities (120, 180, 300, 420, 600 slots), so total_slots is a genuinely informative feature — unlike the previous dataset, which hard-coded a single capacity and therefore taught the model nothing from it.

2.1 Parking lots

Lot	Type	Capacity	Demand character
Tel Aviv University (Ramat Aviv)	university	420	Weekday daytime peaks; follows the academic calendar
Ramat Aviv Mall	mall	600	Evenings & Friday daytime; Saturday-night rush
Tel Aviv Port (Namal)	leisure	300	Weekends, summer, evenings, events
Yarkon Park / Reading	park	180	Weekends and good weather; morning runners
Bavli Residential	residential	120	Inverse commuter pattern (full at night)

2.2 Features & special events

Each hourly record carries calendar features (hour of day, day of week, Israeli weekend flag for Friday/Saturday, month, week of year, season), the lot's capacity and type, weather (temperature, rain), and — most importantly for North Tel Aviv — explicit special-event signals:

- Israeli public holidays (Rosh Hashanah, Yom Kippur, Sukkot, Simchat Torah/Shemini Atzeret, Pesach, Shavuot, Independence Day). On Yom Kippur the whole country stops, so occupancy drops to near zero everywhere.
- Chol HaMoed (intermediate days of Sukkot/Passover) — leisure and mall outings spike.
- Tel Aviv University academic calendar — semesters, exam periods and the summer break strongly shape university-lot demand.
- North Tel Aviv flagship events — Tel Aviv Marathon, Independence Day, 'White Night' (Laila Lavan), the summer beach peak, and Tel Aviv Port festival weekends.

- Winter rain, which suppresses outdoor leisure/park demand but slightly increases mall demand.

Demand is built per lot type from realistic hour/weekday shapes, scaled by these seasonal, holiday, event and weather effects, then made stochastic with hour-to-hour autocorrelation, daily anomaly multipliers and white noise (clipped to a valid 0–100% occupancy).

3. Methodology

Models were trained to predict free_slots from the engineered features (calendar values, cyclical hour/month encodings, capacity, weather and one-hot-encoded lot type / season / academic term / event).

- Time-aware split: the model trains on the first 80% of the timeline and is tested on the most recent 20%. A chronological hold-out is more honest for time series than a random split because it never lets the model 'see the future'.
- Five models plus a baseline were compared: Linear Regression (baseline), Decision Tree, Random Forest, Gradient Boosting, XGBoost, and HistGradientBoosting.
- Metrics: MAE (mean absolute error, in free slots), RMSE (penalises large misses), R^2 (variance explained), MAE as % of average capacity, and 5-fold cross-validated MAE.

4. Exploratory Data Analysis

The data shows clear, learnable structure — exactly the patterns a useful predictor must capture.

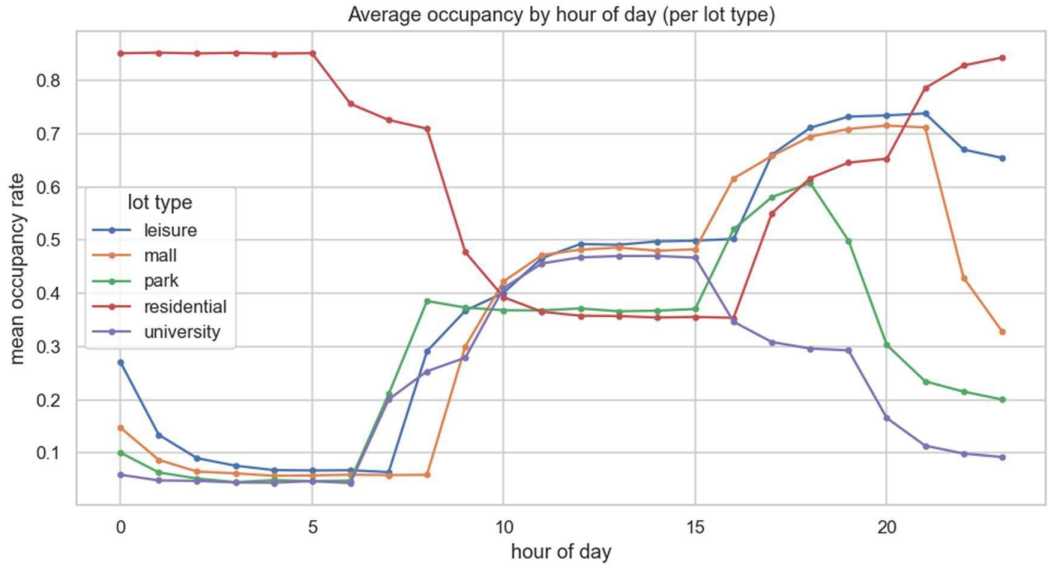


Figure 1. Mean occupancy by hour of day, per lot type.

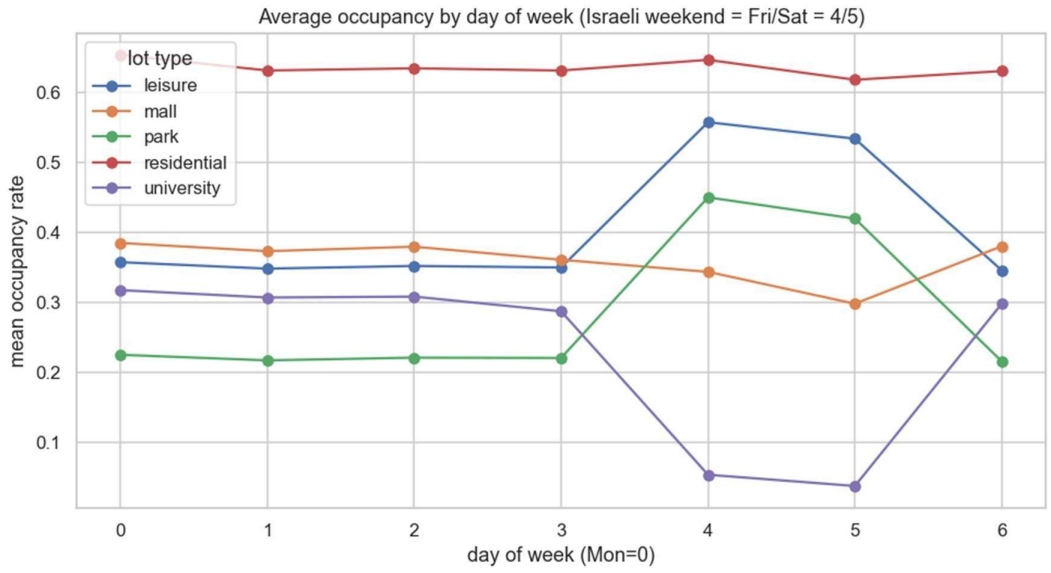


Figure 2. Mean occupancy by day of week (Israeli weekend = Fri/Sat).

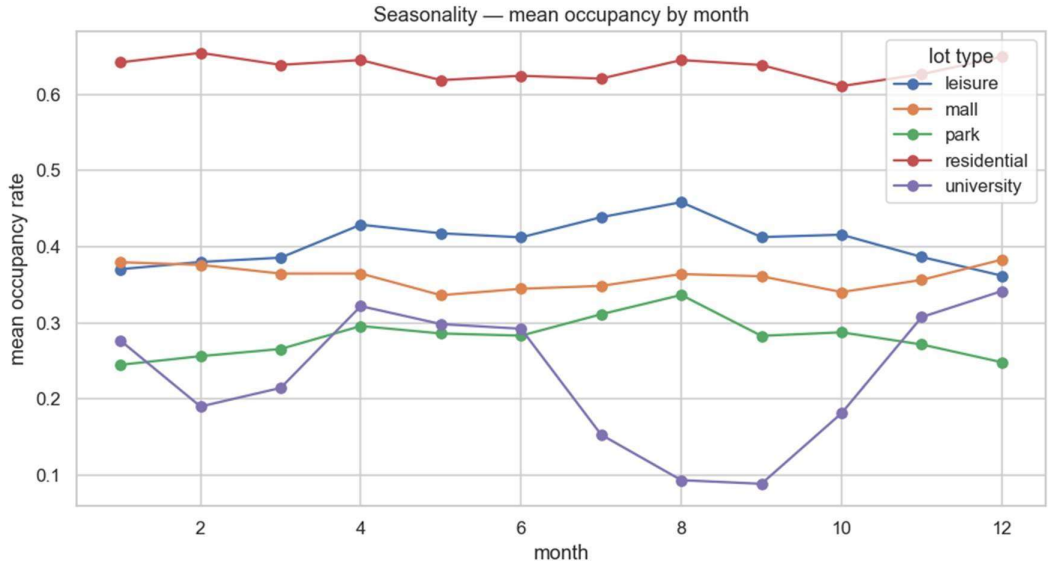


Figure 3. Monthly seasonality of occupancy.

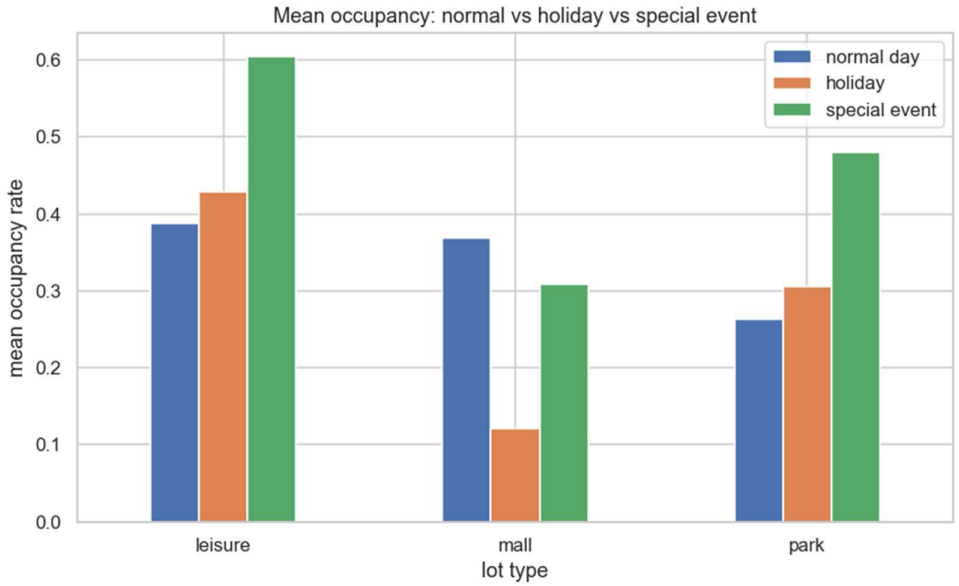


Figure 4. Normal vs holiday vs special-event occupancy.

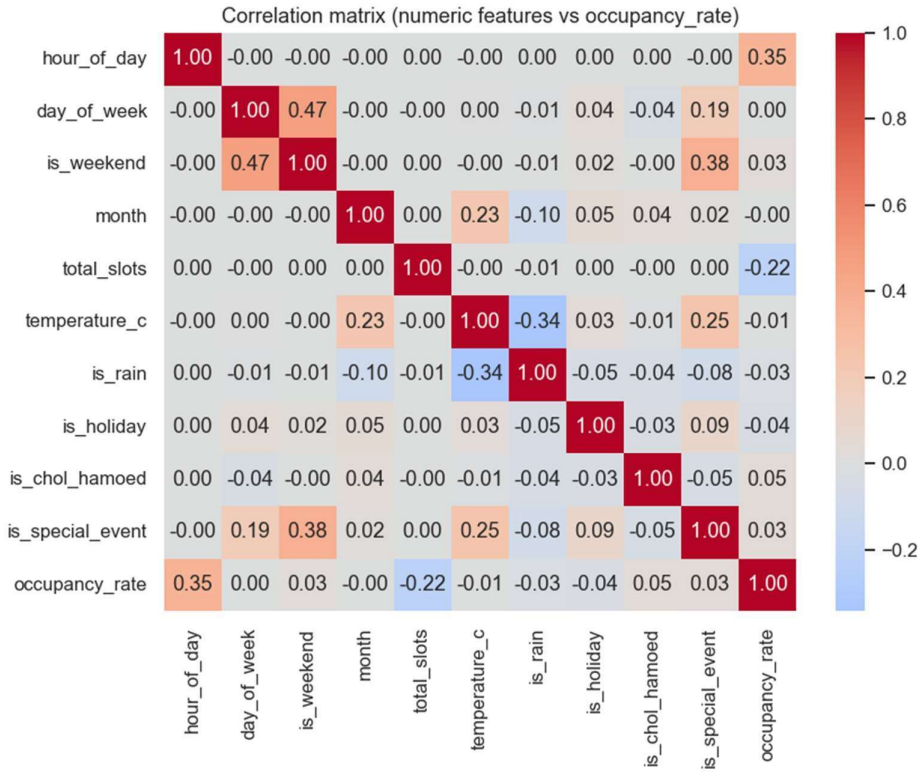


Figure 5. Correlation of numeric drivers with occupancy.

5. Model Comparison

All tree-ensemble models comfortably beat the linear baseline, confirming the relationship between the calendar/context features and availability is non-linear. Results on the time-aware test set:

Model	MAE	RMSE	R^2	MAE (% cap)	CV-MAE
XGBoost	**23.56**	**38.10**	**0.948**	**7.3%**	**15.17**
HistGradient Boosting	23.63	40.41	0.941	7.3%	16.45
Random Forest	24.63	43.09	0.933	7.6%	15.58
Decision Tree	25.83	44.97	0.927	8.0%	17.19
Gradient Boosting	37.57	56.91	0.883	11.6%	36.57
Linear Regression	66.85	86.75	0.729	20.6%	70.19

Table 1. Model metrics on the most-recent-20% test set. XGBoost is the best (bold).

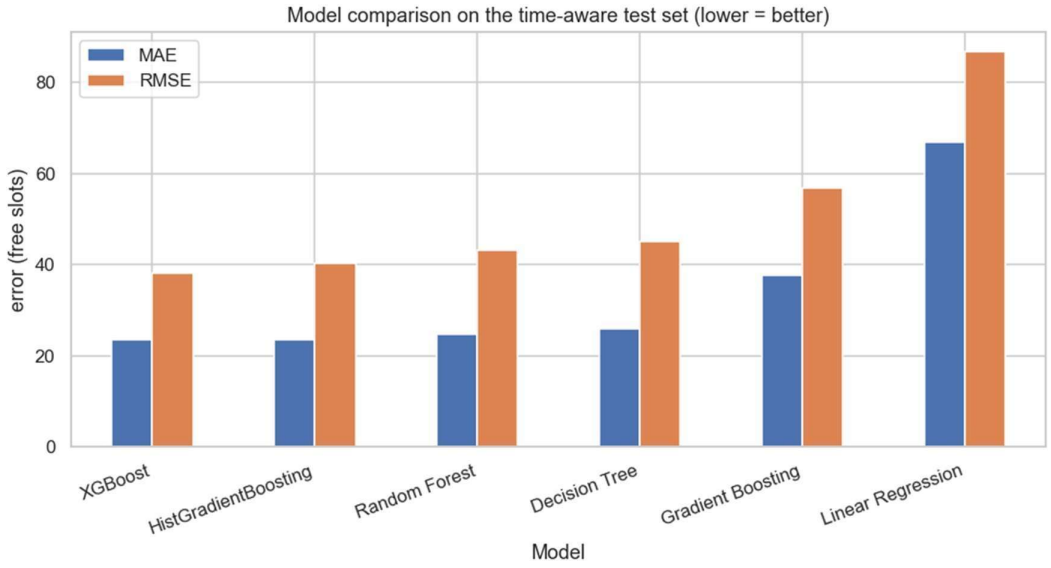


Figure 6. MAE and RMSE per model (lower is better).

6. Best Model — XGBoost

XGBoost achieved the lowest error (MAE 23.56 free slots, RMSE 38.1, R^2 0.948). On average its prediction is within 7.27% of a lot's capacity — accurate enough to drive a 'worth entering?' decision.

6.1 Feature importance

The most influential features are the lot identity/capacity and the calendar context (weekend, academic term), matching the patterns seen in the EDA. Top features: lot_type_residential, lot_type_mall, total_slots, lot_type_university, lot_type_park, lot_type_leisure, is_weekend, academic_term_break.

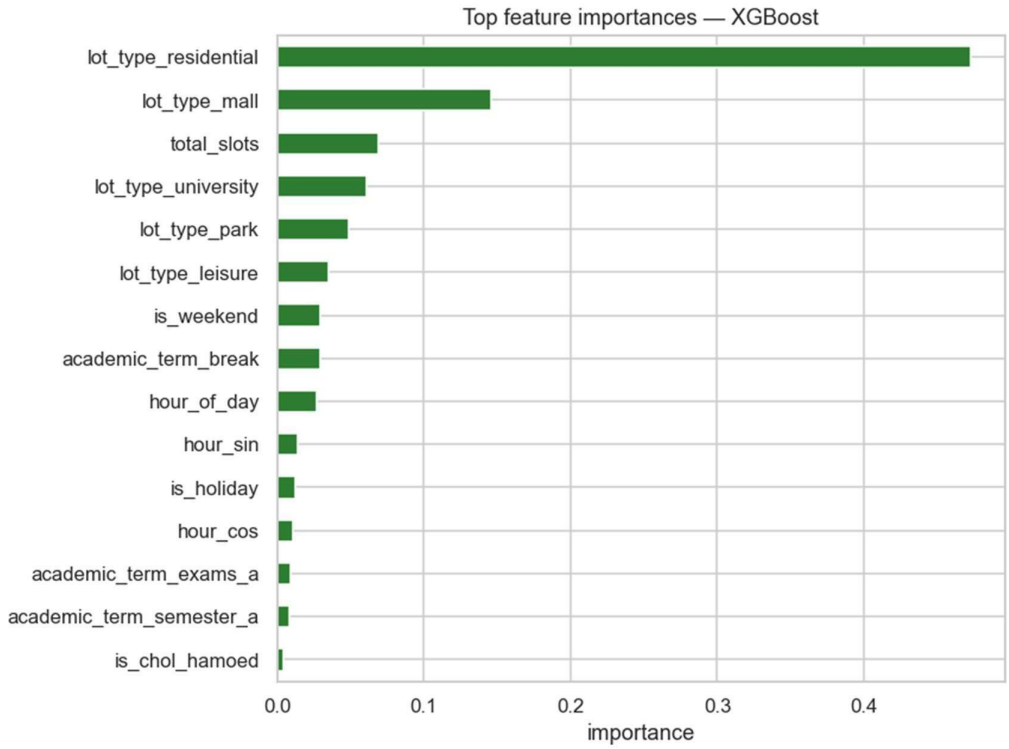


Figure 7. Top feature importances.

6.2 Prediction quality & residuals

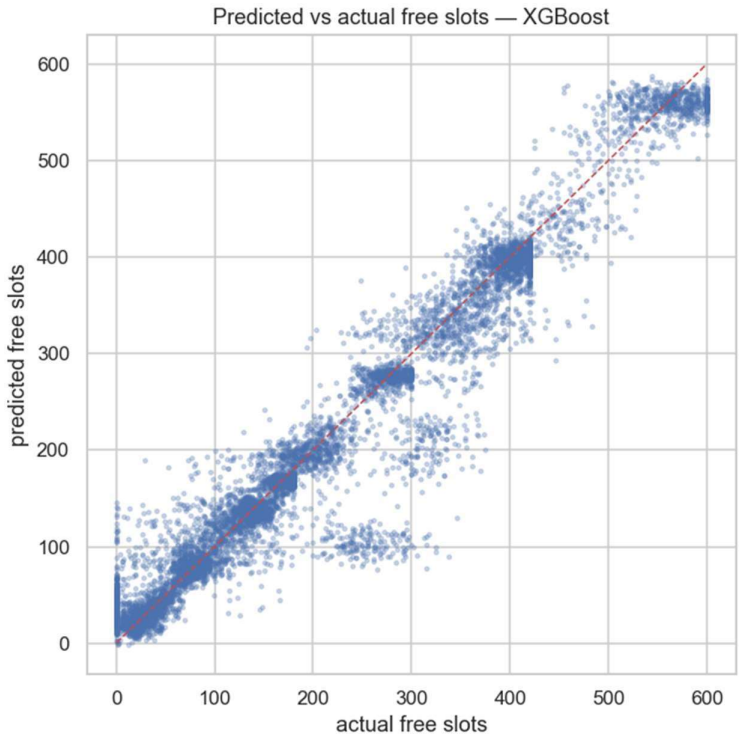


Figure 8. Predicted vs actual free slots (test set).

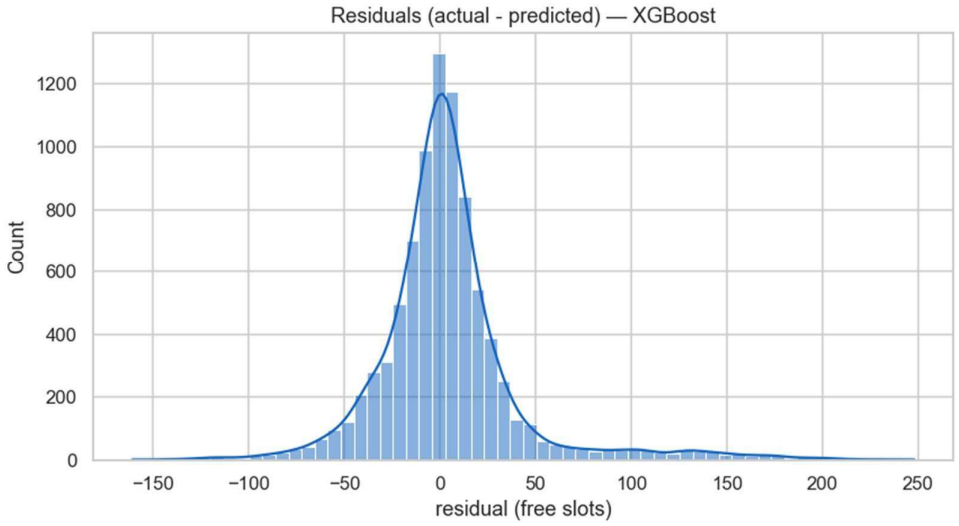


Figure 9. Residual distribution (actual – predicted).

6.3 Hyper-parameter tuning

A randomized search confirmed the model is near its potential. Best parameters: {'subsample': 0.8, 'n_estimators': 300, 'max_depth': 8, 'learning_rate': 0.05}. Tuned test scores: MAE 23.64, RMSE 38.57, R^2 0.946.

7. Conclusion & Recommendation

Recommendation: deploy XGBoost as the parking-availability predictor. It explains 94.8% of the variance in free slots and predicts within ~24 slots on average, far ahead of the linear baseline.

Production note: at inference time the service knows only the target timestamp and the lot's capacity, so the deployed model uses timestamp-derivable features - hour, day of week, weekend, month, day of year, and the Israeli public-holiday flag. It predicts an occupancy rate (capacity-independent, so free = (1 - rate) * totalSlots scales to a lot of any size) and is trained as three quantile models (P10 / P50 / P90), giving a median forecast plus a confidence band. The service exposes /predicts (single point) and /forecast (a full day with the band). Including the day of year makes each date's curve distinct, so the forecast is no longer identical every week. Live weather and ad-hoc event signals are not yet wired into inference and remain a future enhancement; the dataset already supports them, so the model can be extended without new plumbing.

Reproducibility: regenerate the data with data-gen/mockdatagen.py, then run research/model_research.ipynb to reproduce every figure and metric in this report.

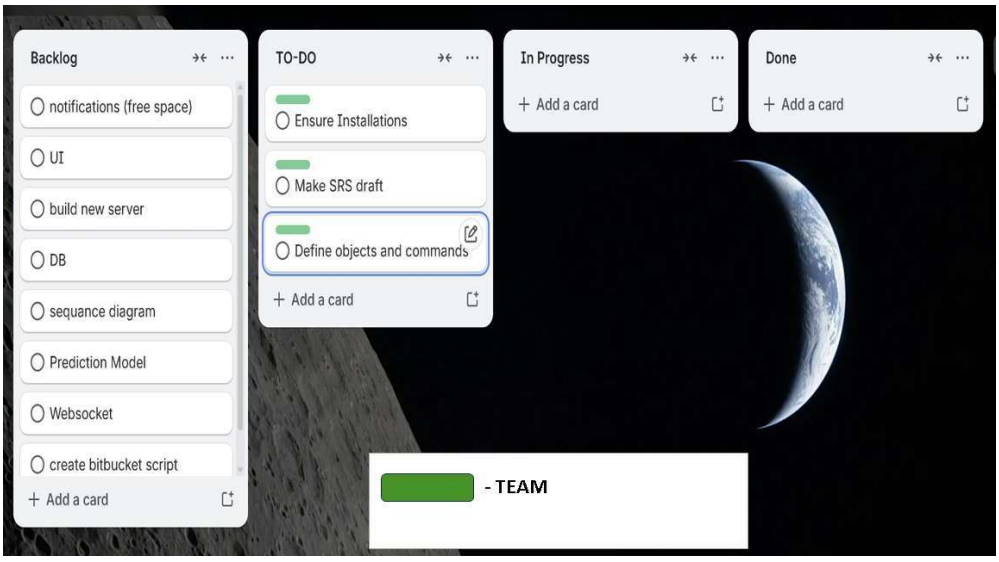
Our Project Progress Throughout Kaban Boards

Important note: The Severity of the tasks is by their order in the todo list.

Sprint 1:

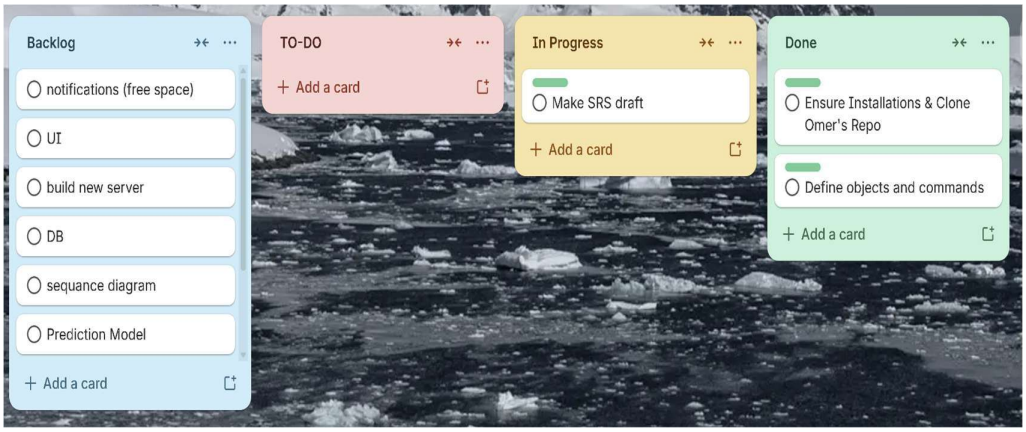
Initial State:

Last Modified: 19/04/2026 13:00



End of Sprint:

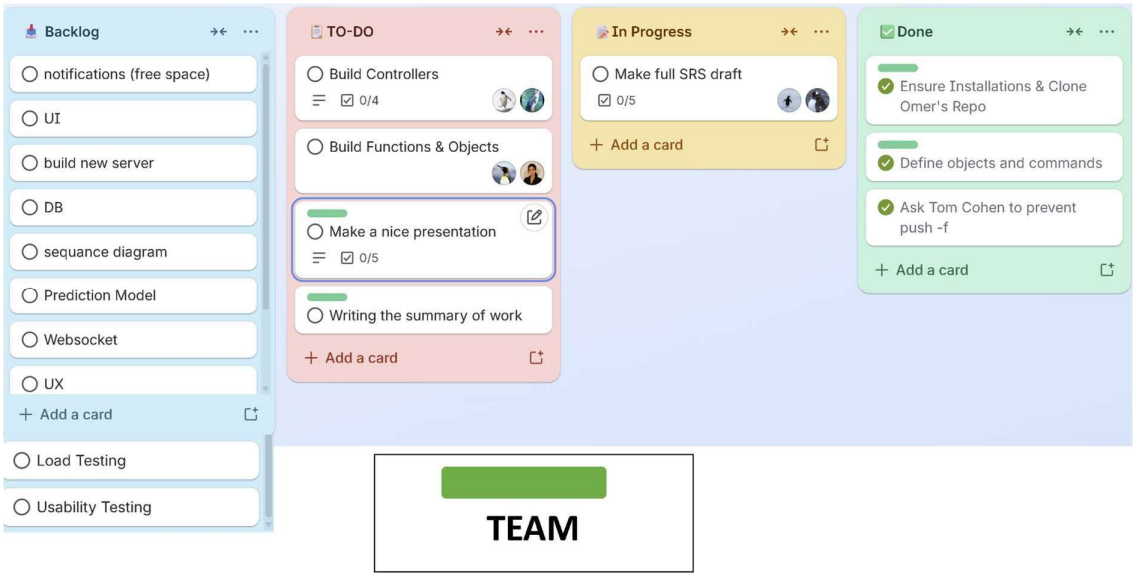
Last Modified: 05/05/2026 23:51



Sprint 63:

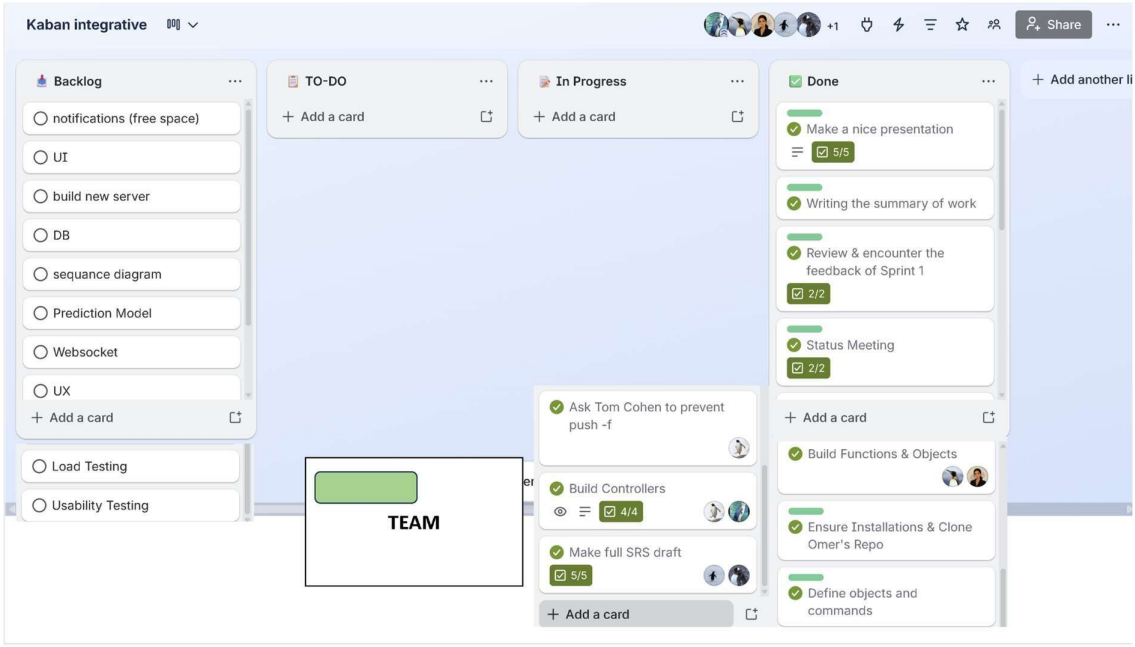
The Project Initiation:

Last Modified: 063/05/2026



End of Sprint:

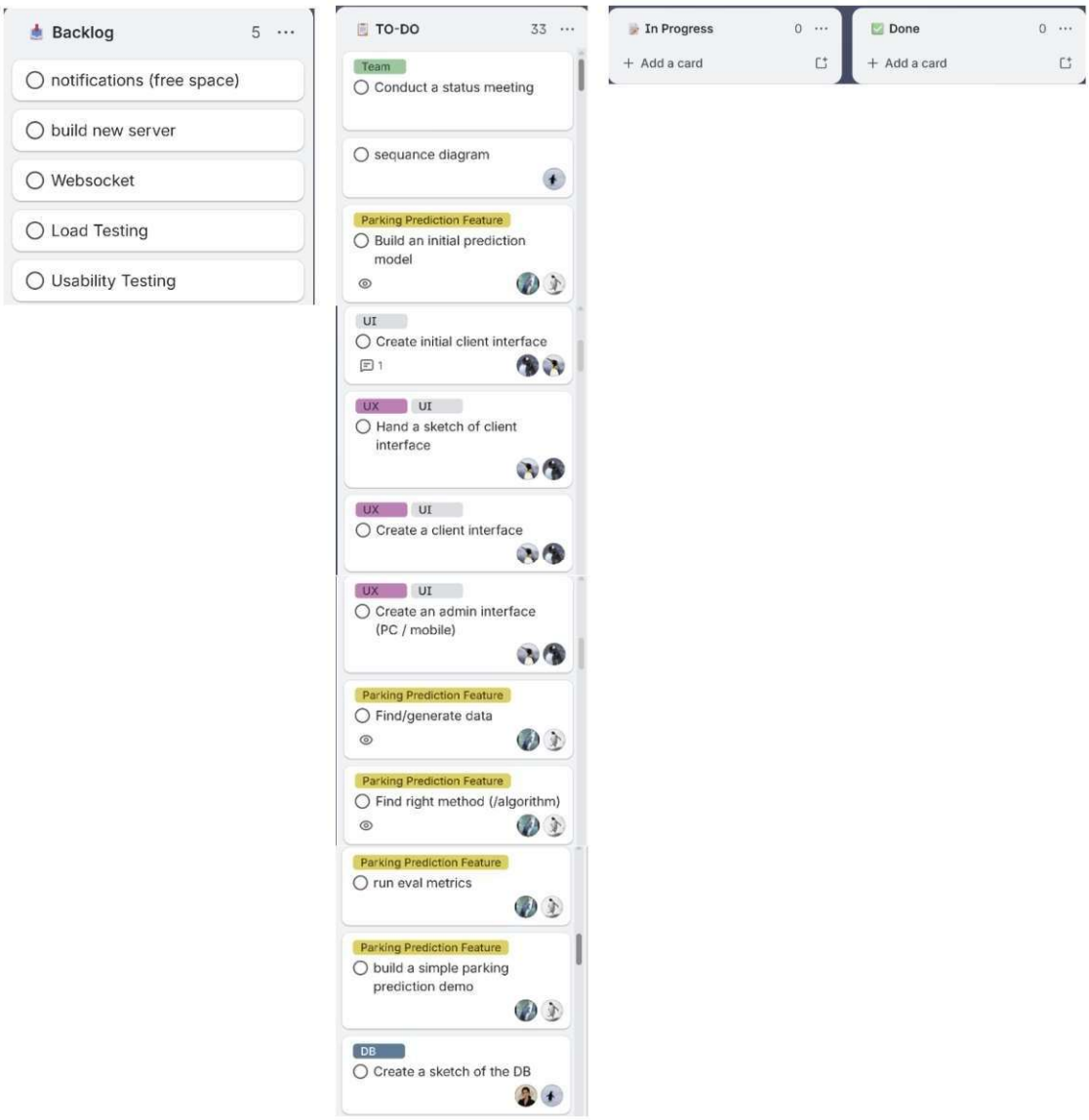
Last Modified: 19/05/2026 23:00



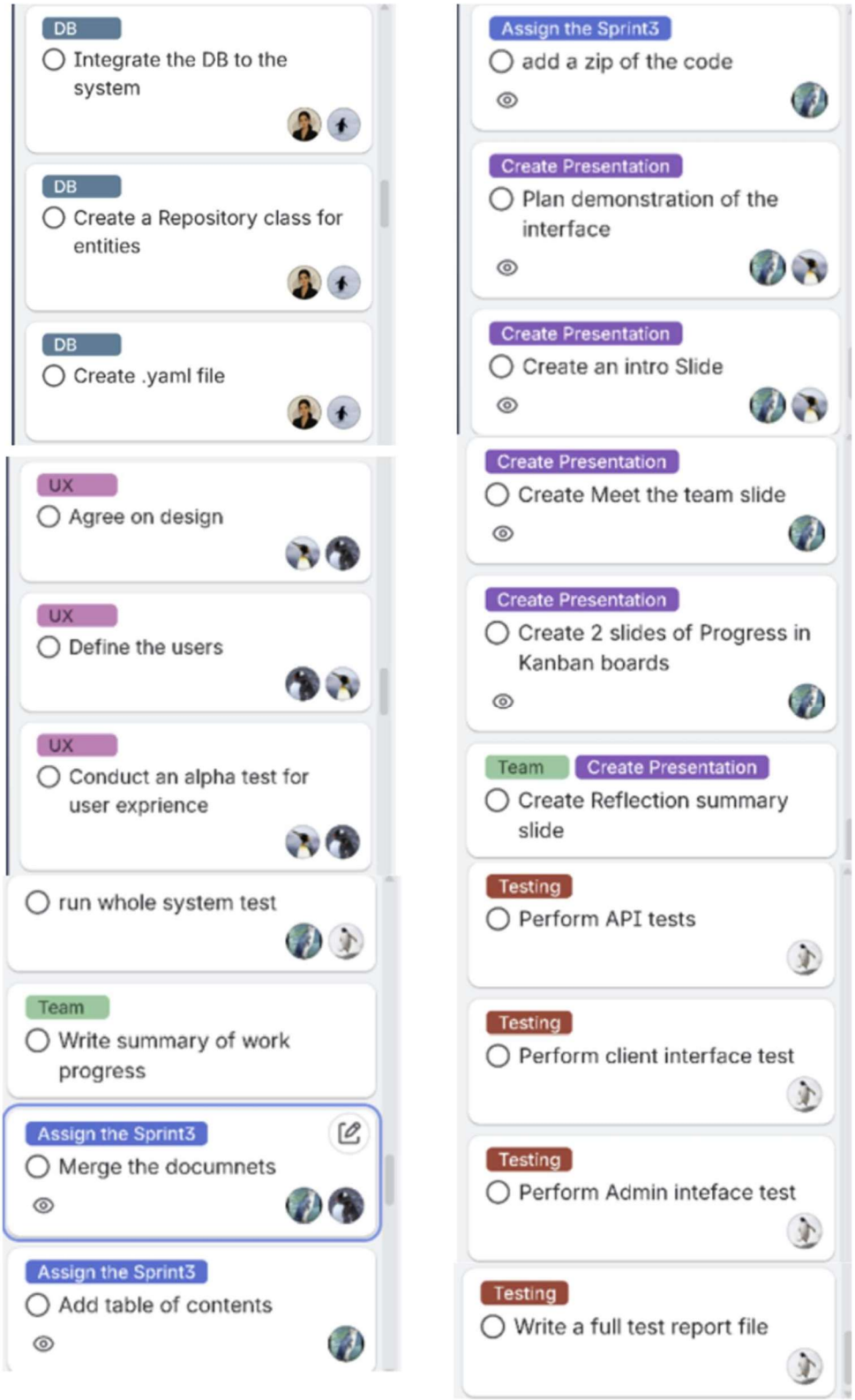
Sprint 64:

The Project Initiation:

Last Modified: 24/05/2026 13:64

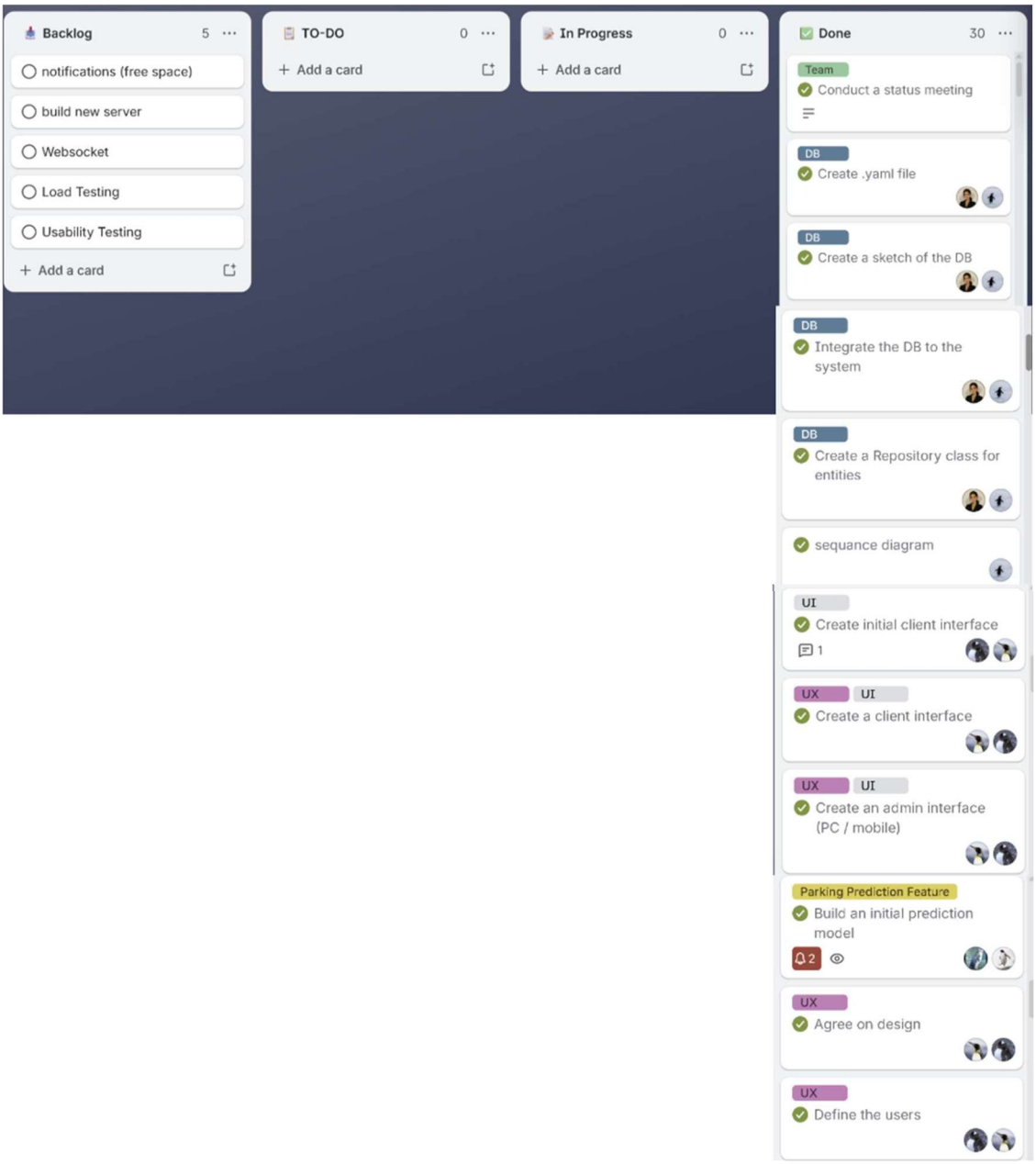


(Continue with the todo list - sprint 3)

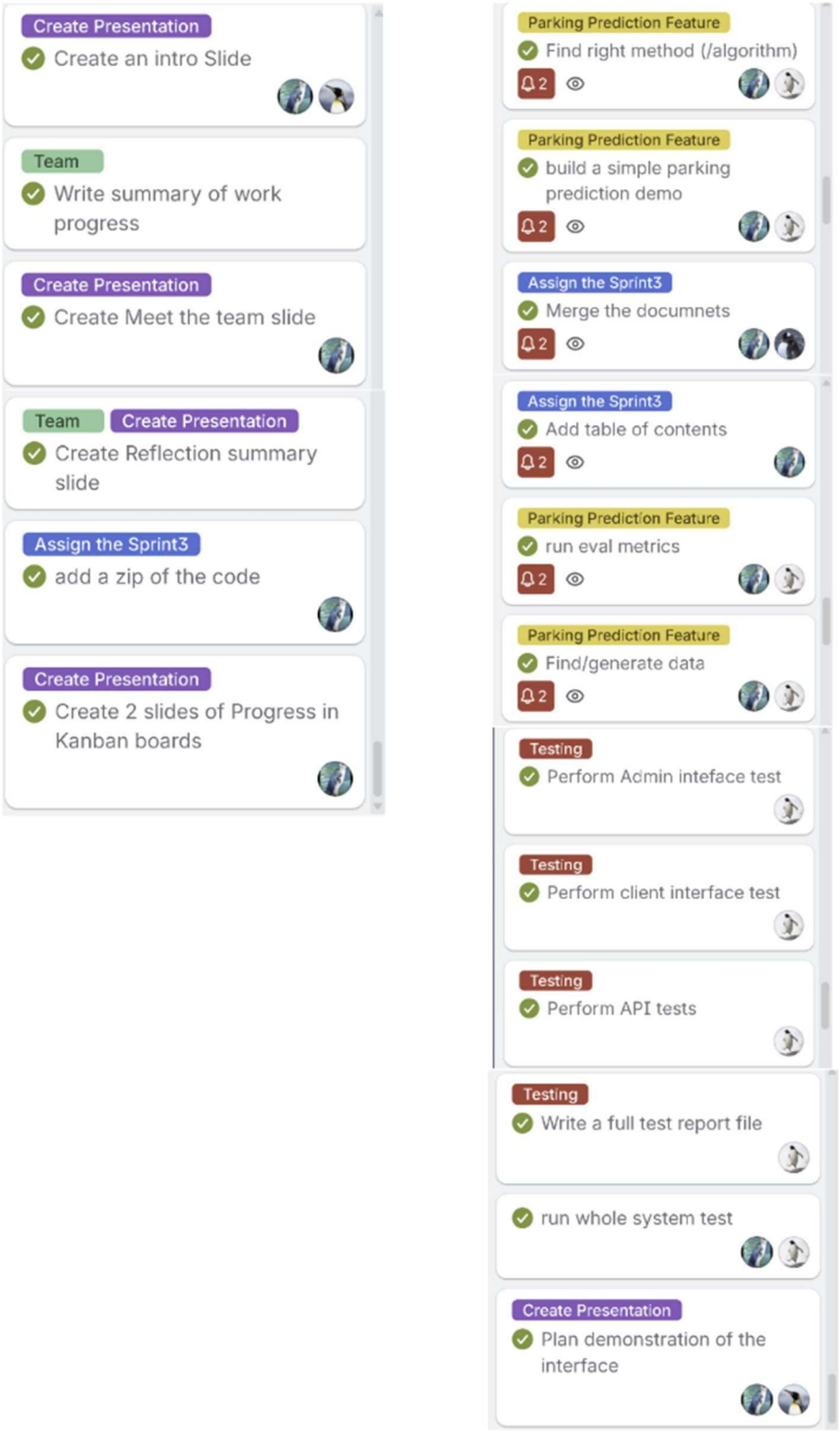


End of Sprint:

Last Modified: 10/06/2026 23:50



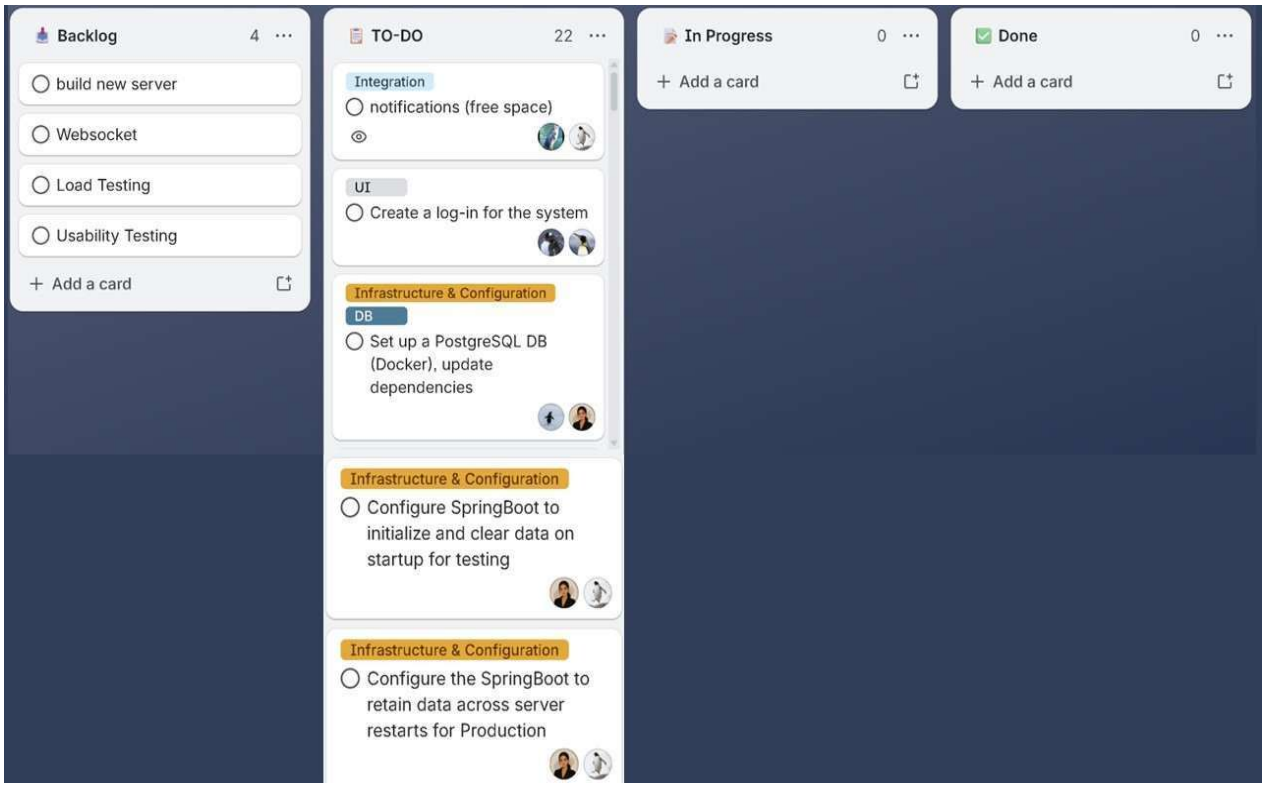
(Continue with the done list - sprint 3)



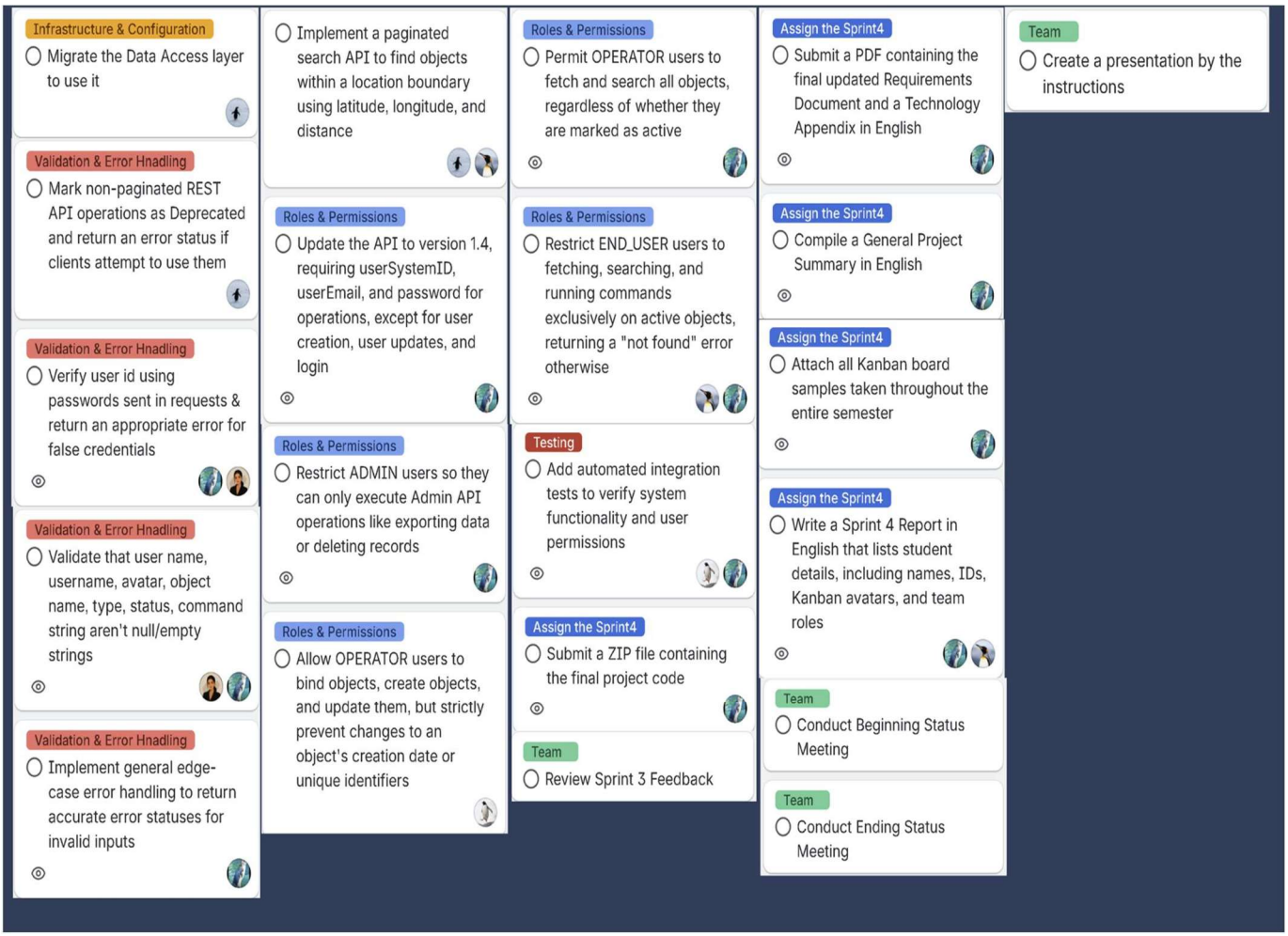
Sprint 4:

The Project Initiation:

Last Modified: 14/06/2026 12:55

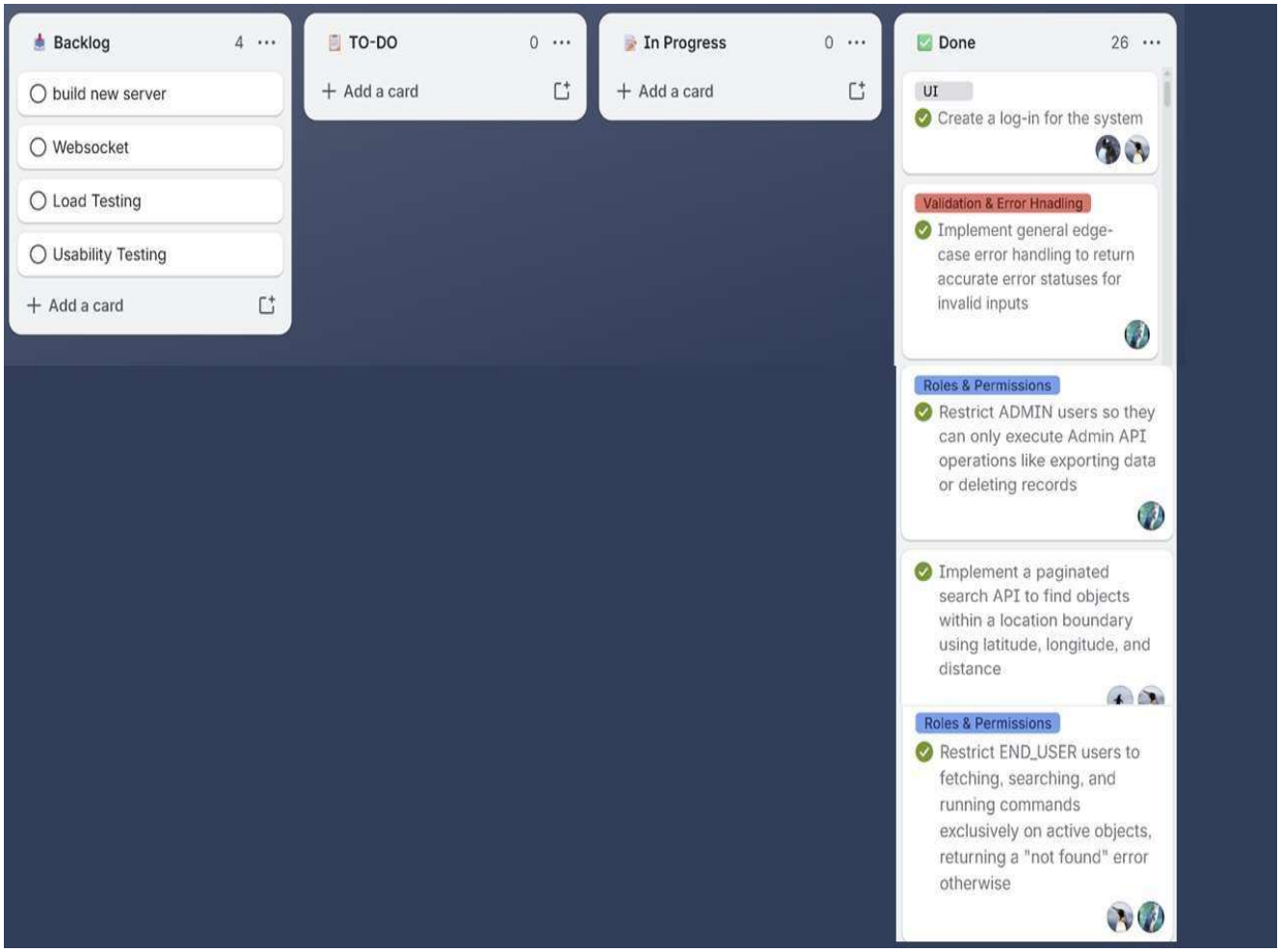


(Continue with the todo list - sprint 4)

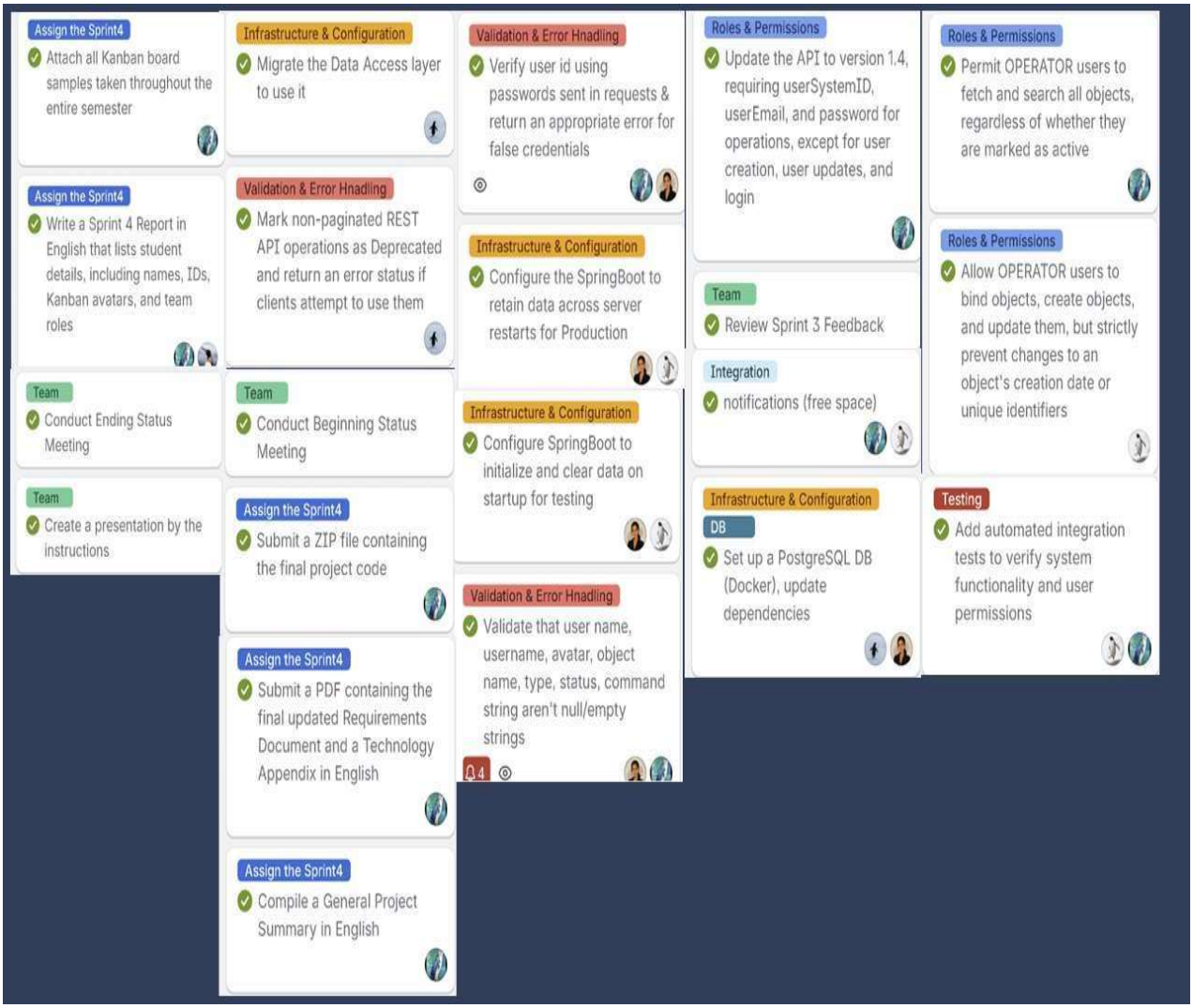


End of Sprint:

Last Modified: 06/06/2026 23:15



(Continue with the done list - sprint 4)



General Summary of Project's Work Process:

To sum up:

- We put our effort and worked hard.
- We learned how to listen and cooperate.
- We adapted a methodology of work: role & small teams.

- **What worked well for your team throughout the semester that you would like to carry over into future industry projects you participate in?**
  - Working in small teams.
  - Separating each major task to smaller tasks.
  - Documenting & journaling our technical work decisions (like the model evaluation report, UX/UI report, etc.)

- **How could you improve your work in future projects you participate in?**
  - Keep updating the Kaban/Gantt on time; especially transitioning between todo, in progress, done.
  - Awareness of team members' feedback, never underestimated team notices.
  - Sticking to our roles, we're all a team but we should know our responsibility.

- **What did you enjoy most about working on the project?**
  - Integrating the system. When we saw the system as a whole we understand the value of our work.
  - Learning new technologies & taking in practice these technologies.
  - Being a part of a team. Again, "we're all a team".

- **What would you do differently if you were to start the project now, given the knowledge and experience you have gained this semester?**
  - We would start first in a profound research of the users of our product, and then focus on the SRS.
  - We would start in documenting, then head off work.
  - We would understand the tradeoffs of each team member, and learn the needed skill to contribute to the team work.

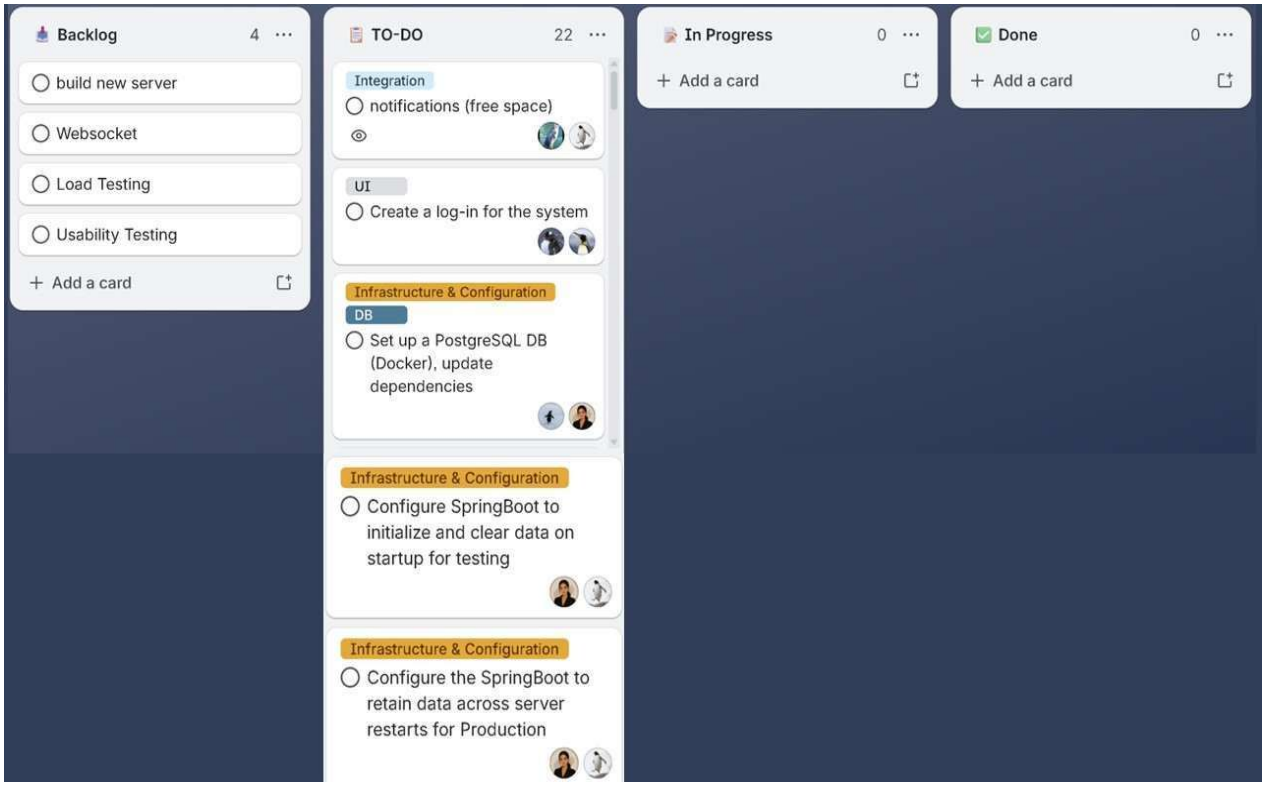
- **Was the teamwork also conducted remotely, in addition to face-to-face meetings?**
  - We used the following tools:
    - Teams for meeting
    - Bitbucket for version control
    - Trello for updates in work process
    - A shared Google Drive for the research & progress reports
    - Canva for a shared work system of presentation editing.
    - Outlook for notification of Trello.
  - We were focused on the following method: in the beginning status meeting, and continued updates through our tools.
  - The work remotely gave us flexibility in schedule, but made us document more & keep track with the changes in ourselves.

Progress Throughout The Kanban Board - Sprint 4:

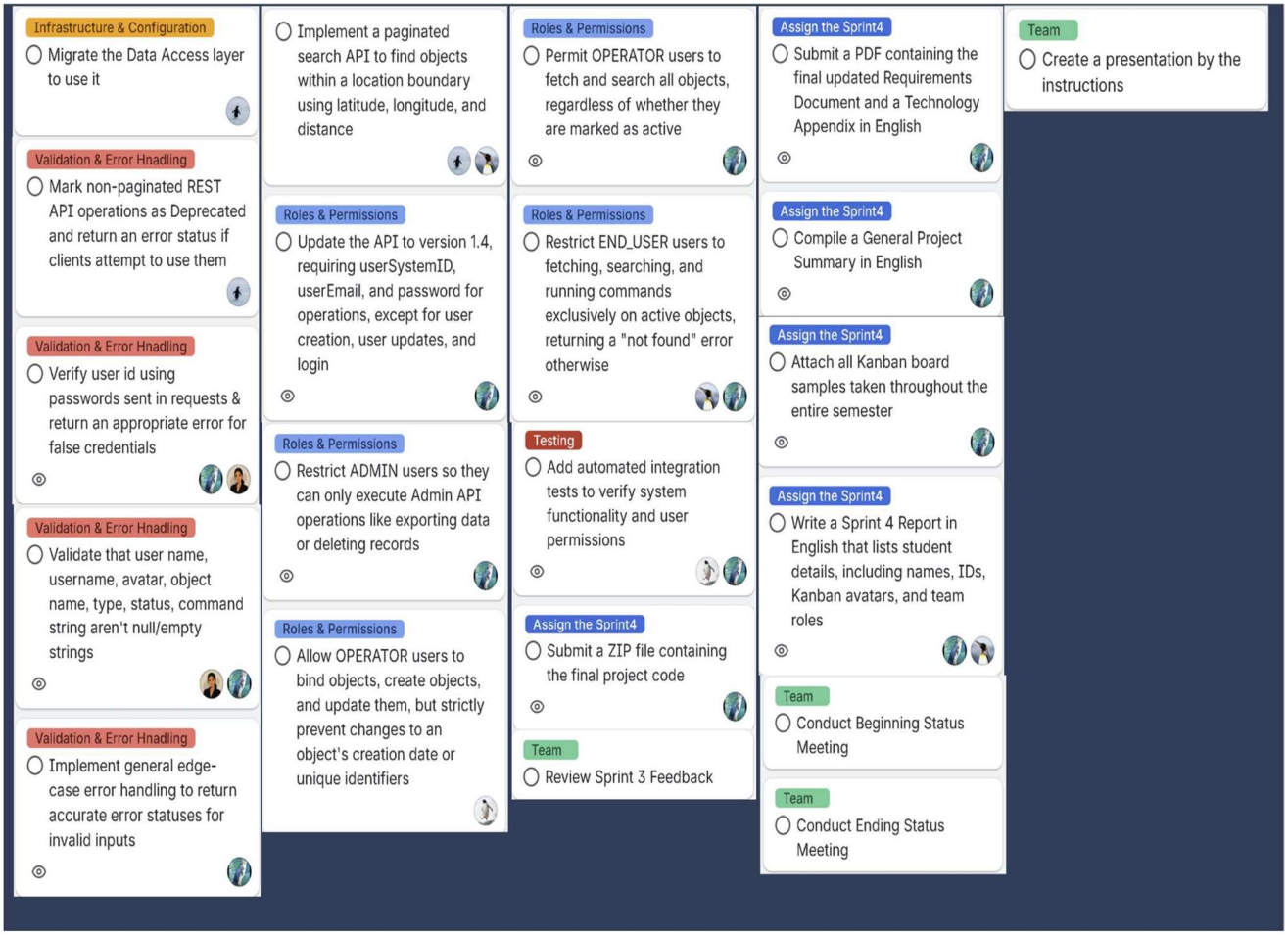
Important note: The Severity of the tasks is by their order in the todo list.

The Project Initiation:

Last Modified: 14/06/2026 12:55

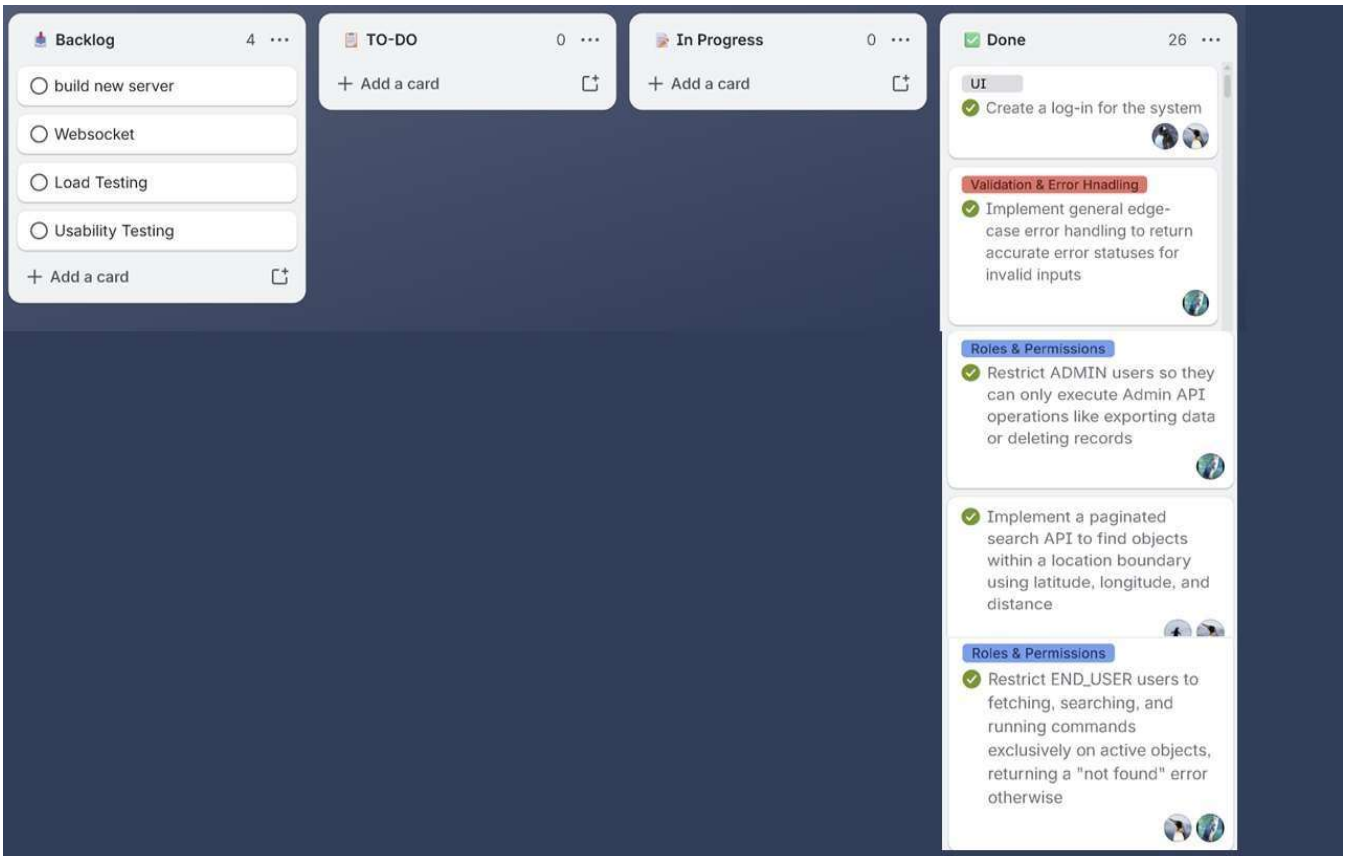


(Continue with the todo list - sprint 4)

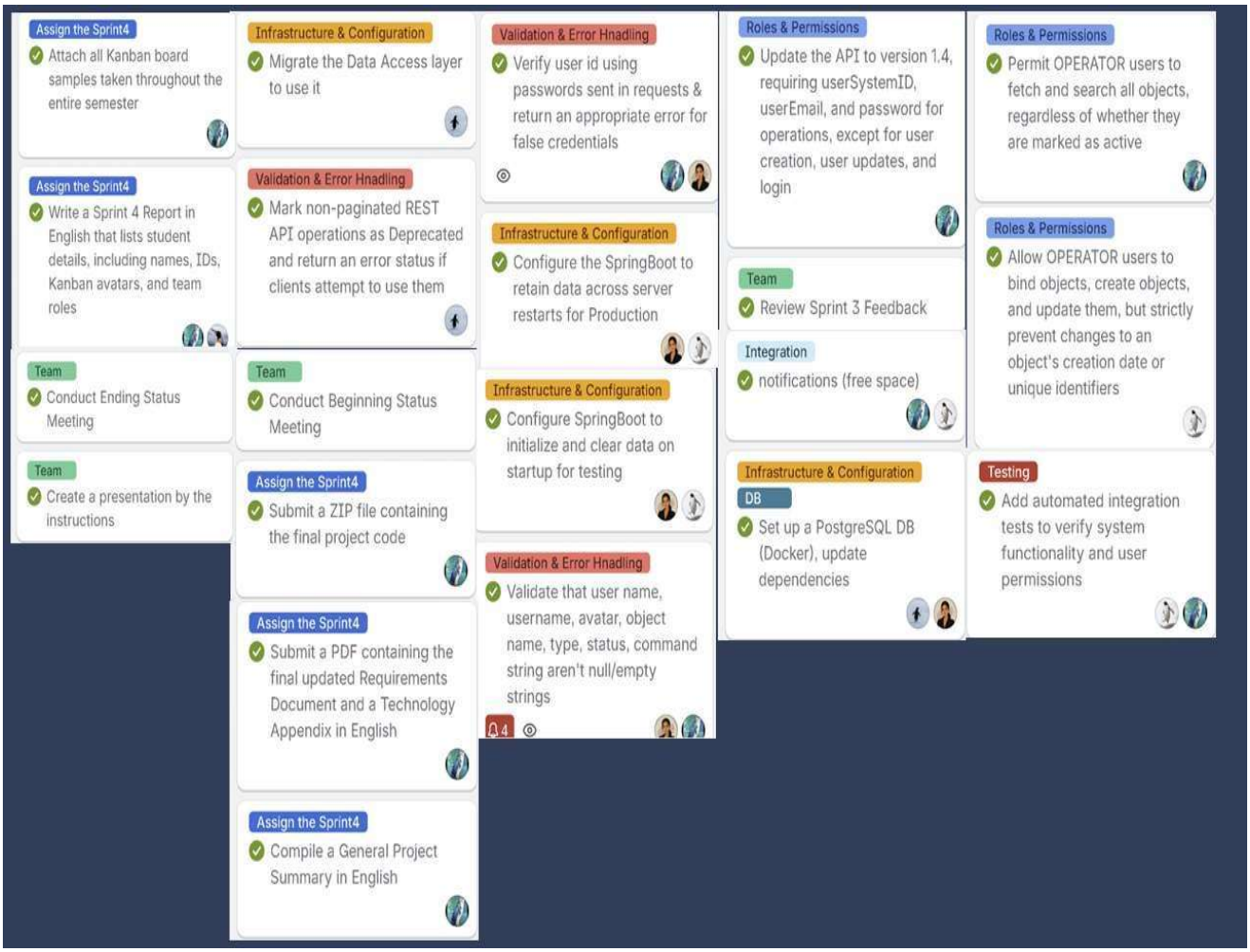


End of Sprint:

Last Modified: 06/06/2026 23:15



(Continue with the done list - sprint 4)



General Summary of Work in Sprint 4:

To sum up:

- Working restlessly to get the best project done.
- Keeping up with proficiency (documented & structured work process).
- Persevering working in small teams (even in solo).

- **What went well for the team and should be continued on future projects?**
  - Collaborating in small teams as part of a bigger team.
  - Async work model
  - Awareness of the updates in the Kanban board.

- **What could be improved in team work?**
  - Awareness of each other's notes.
  - Viewing all the changes in documentation.
  - Keep updates on real time (improved from last sprint).

- **What problems did the team encounter through this phase of work?**
  - Learning how to handle complex actions in relational DB.
  - Staying organized documentation.
  - Confusing with the recent feedback (same notes understand differently for some team members)

- **Why did we not complete all planned work?**
  - We finished all the planned work, but we left the backlog still with enormous tasks that we would like to do.